

**Szegedi Tudományegyetem
Informatikai Intézet**

SZAKDOLGOZAT

Gyurcsó-Licsman Péter

2026

**Szegedi Tudományegyetem
Informatikai Intézet**

**Ébredő Világok, az online szerepjáték
platform**

Szakdolgozat

Készítette:

**Gyurcsó-Licsman
Péter**

Programtervező informatikus BSc
szakos hallgató

Témavezető:

**dr. Jánki Zoltán
Richárd**

egyetemi adjunktus

**Szeged
2026**

Feladatkiírás

A hallgató feladata az asztali szerepjátékok virtuális térben történő elterjedésének vizsgálata és egy olyan egyedi alkalmazás létrehozása, ami az *Ébredő Világok* szerepjátékot helyezi online térbe.

Az alkalmazás fő feladata a játékmester és a játékosok támogatása az általuk készített karakterek és kalandok tárolásával, a látogatók tájékoztatása a szerepjáték szabályairól és egy olyan felület biztosítása, amin a közösség tagjai a szabályzat félautomatikus rendszerét használva játszhatnak a Dungeons & Dragons Beyond alkalmazás mintájára egy Angular – Firebase alapú környezetben.

Tartalmi összefoglaló

A dolgozat célja egy webes alkalmazás implementálása a még fejlesztés alatt álló *Ébredő Világok* asztali szerepjáték számára, lehetőséget kínálva az érdeklődőknek a játék világának megismerésére, a szabályok elsajátítására és ezek gyakorlati alkalmazására egy online játék felületen.

A virtuális szerepjátás a pandémia időszaka alatt terjedt el leginkább, mivel az emberek bezártsága alapvetően akadályozta a csoportos gyülekezéseket, ami elengedhetetlen a szerepjátékoknál.

A legismertebb szerepjáték a Dungeons & Dragons, mely egy epikus fantázia stílust képvisel – hasonlóan *A Gyűrűk Urához* és a *Trónok harcához*. Ez egy nagyon sokrétű és nagy szabadságot kínáló játérendszer, aminek megvan a saját webes megoldása, ahol megtalálhatóak leírások a rendszerről, útmutatók a karakter- és kampány készítéshez, illetve lehetőség arra, hogy a játékosok végig játszassanak egy kampányt. Emellett léteznek általános célú platformok is, melyek támogatják ezt a rendszert és még sok másikat is. Ilyen például a Roll20 és a Foundry VTT.

Az alkalmazás elkészítése egy komplex feladat, mivel egyszerre kell digitális szabálykönyvként, útmutatóként karakter- és kampánylapként működnie, miközben interaktívan segíti a játékmestert és a játékosokat is. Valós idejű kommunikációt folytat az adatbázissal, automatizált mechanikákat (pl. harc) támogat és a játékmester és a játékosok közötti kommunikációt is biztosítja.

Ez egy kliens-szerver architektúrára épülő, szolgáltatásorientált alkalmazás elkészítésével valósult meg. A frontend oldalról Angular keretrendszerrel, backend felől Google Firebase Firestore adatbázissal készült, mely a valós idejű adatáramlást és ebből kifolyólag gördülékeny játékelményt biztosítja. A Firestore Security Rules-zal kifejezett biztonságot kínál szerveroldalról, az azonosítást a Firebase Authentication végzi. Az online beszélgetést a WebRTC technológiára épülő PeerJS könyvtár működteti, hogy a játékosok folyamatosan kommunikálhassanak. A 3D-ben megjelenő kockadobást ThreeJS és Cannon-es könyvtárak valósítják meg.

Az elkészített alkalmazás nagyjából teljes értékű reprezentációja az *Ébredő Világok* szerepjátéknak, azonban egyes leírások még hiányoznak, ezért ezeket a részeket még nem lehet publikálni.

Tartalomjegyzék

FELADATKIÍRÁS.....	3
TARTALMI ÖSSZEFOGLALÓ.....	4
TARTALOMJEGYZÉK.....	5
BEVEZETÉS.....	6
1. VIRTUÁLIS SZEREPJÁTÉK PLATFORMOK.....	7
1.1. D&D BEYOND	7
1.2. ROLL20	9
1.3. FOUNDRY VTT	10
2. FUNKCIONÁLIS SPECIFIKÁCIÓ ÉS RENDSZERTERV	12
2.1. KÉPERNYŐTERVEK	12
2.2. SZEREPLŐK ÉS FUNKCIÓIK	22
3. HASZNÁLT TECHNOLOGIÁK.....	23
3.1. BACKEND	23
3.2. FRONTEND	24
3.3. WEBES AUDIOKOMMUNIKÁCIÓ	25
4. AZ ALKALMAZÁS MAGAS SZINTŰ FELÉPÍTÉSE.....	25
4.1. KLIENS KÖRNYEZET	25
4.2. PEERJS SIGNALLING KÖRNYEZET.....	26
4.3. BACKEND KÖRNYEZET	27
5. MODULOK ÉS KOMPONENSEK	28
5.1. A LOGIKAI RÉTEGEK	28
5.2. SZOFTVERES KAPCSOLÓDÁSI PONTOK	30
6. ADATMODELL	32
6.1. FELHASZNÁLÓ	32
6.2. KARAKTER	32
6.3. KALAND	33
6.4. FÓRUM	34
7. MAGAS SZINTŰ FOLYAMATOK.....	35
7.1. BEJELENTKEZÉS AZ OLDALRA	35
7.2. JÁTÉKHOZ CSATLAKOZÁS	35
7.3. FIÓK TÖRLÉSE	36
8. FONTOSABB KÓDRÉSZEK	37
8.1. KARAKTEREK / KALANDOK LÉTREHOZÁSA	37
8.2. CSATLAKOZÁS A JÁTÉKHOZ	39
8.3. TÁMADÁS ÉS REAKCIÓ.....	46
9. TAPASZTALATOK, TOVÁBBFEJLESZTÉSI LEHETŐSÉGEK.....	51
9.1. TOVÁBBFEJLESZTÉSI LEHETŐSÉGEK	51
9.2. TAPASZTALATOK	51
IRODALOMJEGYZÉK.....	53
NYILATKOZAT	54
NYILATKOZAT AZ MESTERSÉGES INTELLIGENCIA HASZNÁLATÁRÓL	55
MELLÉKLETEK.....	56

Bevezetés

Az *Ébredő Világok* egy évfolyamtársam által fejlesztés alatt álló szerepjáték[20], aminek tesztelési és finomhangolási folyamataiban az elmúlt években aktívan részt vettem. A rendszer fejlődése két évre visszamenőleg jutott el egy használható és játszható szintre. Maga a szerepjáték még nem publikus, így a dokumentációját a saját Google Drive tárhelyemből hivatkoztam.

A fejlesztési folyamatot beindító ok egy beadandó dolgozat volt, amit az említett évfolyamtársam készített a szerepjáték világában élő három fajról. A dolgozat elkészítésének szemtanúja voltam és a folyamat közben egy későbbi teljesértékű digitalizációról értekeztem vele, amit ő is támogatott.

A vizsgaidőszak kezdetével arányosan megváltozott a szabadidőm. Az első elkészített prototípus egy PHP backend és MariaDB adatbázist használó webalkalmazás volt egy karakterkészítési funkcióval, amit a vizsga időszak alatt készítettem el. Ez az alkalmazás az asztali szerepjátékok egyik gyengeségét, a rengeteg papír használatát és azok elvesztését volt hivatott ellensúlyozni. Ez a folyamat világított rá a tényre, hogy webfejlesztéssel szeretnék foglalkozni.

A második prototípus a következő félévben készült Angular keretrendszerrel és a Google Firebase szolgáltatásainak használatával. Ez a megvalósítás semmiben sem különbözött az első prototípustól funkcionális téren. A fejlesztési folyamat egy másik rendszer keresésére szolgált.

A teljes értékű implementáció eléréséhez az alkalmazásnak a teljes szabályrendszert, a világ történetét, a világban található élőlényeket, a tájak leírását és egy teljes világtérképet is magába kell foglalnia. Emellett támogatni kell a szerepjáték gyakorlati játéklehetőségét valós idejű adatszinkronizációval. Ennek az alkalmazásnak a megvalósítására a második prototípus tökéletes tapasztalatnak és kiindulási pontnak bizonyult.

1. Virtuális szerepjáték platformok

Mivel az *Ébredő Világok* egy teljesen egyedülálló szerepjáték rendszer, ezért bár vannak hasonló mechanikák a többi platformhoz képest, nem lehet csak úgy összemosni valamelyik már létezővel.

1.1. *D&D Beyond*

Tekintettel arra, hogy a teljes szerepjáték publikálása volt a cél, fő mintául a D&D Beyond szolgált, ugyanis ez az alkalmazás megvalósítja azt a szerkezetet, amit én is követni szerettem volna. Egyszerre egy digitális gyűjtemény a szerepjáték rendszeréből, szabályokkal, ismertetőkkal és leírásokkal a játék világának fajairól, jelenségeiről, tárgyairól és egy környezet, mely biztosítja, hogy egy légtérben tartózkodás nélkül is élvezni lehessen a játékot. Mindezt a szerepjáték hivatalos kiadójaként.

A Dungeons & Dragons Beyond eleinte nem egy virtuális szerepjáték platformnak indult, hanem egy digitális szabálykönyvnek és egy online karakterkészítő szoftvernek. Magával a szerepjáték játszhatóságával nemrégén kezdtek el foglalkozni.

A rendszerhez tartozó elemek külön-külön oldalra csoportosítva vannak kilistázva. Mindegyik listaelemhez tartozik egy átfogó, pár mondatos leírás, vizuálisan a fogalommal összeköthető ábrázolás és attól függően, hogy fajt, szerepet, tárgyat, stb. listáz ki az oldal, egyéb jellemzőket látni.

A legfontosabb és legkifinomultabb funkció az egész oldalon a karakterkészítés. A rendszer rengeteg segítséget kínál a felhasználónak a folyamat során. A karakterek létrehozását egy *tutorial* támogatja, ami lépésről-lépésre elmagyarázza a folyamat részeit és plusz segítséget, tanácsot, ajánlásokat nyújt. Nagyon sok tulajdonság létezik, így eléggé hosszadalmas tud lenni egy karakter elkészítése, ezért felhasználó választhat, hogy teljesen tiszta lappal indul, esetleg egy már előre összeállított, játékra kész karaktert választ ki. Van lehetőség egy teljesen véletlenszerű karaktert generálni, ahol választható a karakter faja, szerepe és megadható a neve, de nem kötelező. Illetve, az előző tulajdonságok megadásával egy gyors karakterépítő funkció is elérhető, amely lényegében ugyanaz, mint a véletlenszerű karaktergenerálás.

A kampány készítés üresnek érződik. Az egész 4 mezőből áll, az első a név mező, ahol el lehet nevezni a kampányt, a többi pedig leírások. Természetesen a kampány leírása kifejezetten fontos és amíg nem volt a D&D Beyond-nak saját virtuális platformja a játék lehetőségére, addig teljesen elfogadható lett volna. Így pedig hiányolom belőle azt, hogy fel tudjam építeni a helyet, ahol a játékosok tudnak játszani. Ehelyett ehhez egy teljesen különböző alkalmazást kell használni, amiben csak magát a térképet lehet megrajzolni, a karakterek – amikkel a játékosok találkozhatnak – csak utólag adhatók hozzá.

A fórum részen teljesen érthető fórum-alfórum struktúrát követ az oldal. Megtalálhatóak általános fórumok, főleg a játék állapotára irányulva, *release issues*, *bugs & support*, *news & announcements*, *general discussion*, stb. A különböző szerepekhez is tartoznak alfórumok, ahol az adott szereppel kapcsolatos gondolatokat osztják meg a játékosok egymással.

A D&D Maps a szerepjáték virtuális platformja, ahol lehetőség van játszani magával a játékkal. A játék indítása úgy néz ki, hogy játékmester kiválaszt egy kampányt, ekkor betöltődik a kampányhoz köthető térkép. Ezután a kampány készítő részről kihagyott egyéb karakterek és lények lehelyezése itt lehetséges. Ezen a felületen lehet meghívni játékosokat egy csatlakozási link segítségével. Ezt a linket bemásolva a játékosok egy várakozó képernyőt látnak, amíg a játékmester el nem indítja a játékot. Maga a játék a térképen folyik, ahol egy négyzetrácsos területen lehet mozgatni a tokenekként megjelenített karaktereket és lényeket. Érdekes módon ezt mind a játékmesterként és a játékosként is megtehetem, ami egyáltalán nem optimális, mivel egy sárkányt egy játékosnak nem szabadna mozgatnia. Emellett nincs közvetlen hozzá férése a játékosoknak a karaktereihez, mivel az egy külön oldalon található.

Meg kell említeni a Demiplane oldalt, mely az egykori D&D Beyond fejlesztőinek keze munkája, miután elhagyták a céget, mely a mai nap is fejleszti azt. Az elképzelés ugyanaz. Összegyűjtötték a Dungeons & Dragons szerepjáték leírásait. Nagyon sokáig csak így működött az oldal, mint egy adatbázis a játékról. Azonban a D&D mellett más szerepjátékok is megtalálhatóak, mint például a Pathfinder, Daggerheart, Marvel Multiverse, Fallaout és még sok más. Ezeket bizonyos NEXUS-okba rendezték, melyek igazából a szerepjátékok saját mellékoldalaiként szolgálnak. Karakterkészítésre és játékokra viszont még nem alkalmas a Dungeons & Dragons világot tekintve.

1.2. Roll20

A Roll20 a D&D Beyond-hoz viszonyítva visszafelé járta meg a fejlődés útját. Eleinte csak virtuális platformként működött majd utólagosan hozták be a gyűjtemény részeket a különböző szerepjátékokhoz (D&D, Pathfinder, stb.). Ez az egyik legrégebbi, legnépszerűbb és leginkább hozzáférhető webalkalmazás, amely a virtuális szerepjátszás történetében létezik.

Az alkalmazás hasonlóan támogatja a karakterkészítést, mint a Dungeons & Dragons Beyond, bár kevesebb tippel és ajánlással, ráadásul játékba csatlakozás után lehet csak elkészíteni a karaktert. Ez alapvetően első gondolatra nem tűnik a legjobb formátumnak, de a Roll20-nek előnye a D&D Beyonddal szemben, hogy támogatja a hang- és videóhívást, ami, ha nem használnak a játékosok egy külön alkalmazást beszélgetésre, és esetleg egy kezdő is szerepel a jelenlévők között, akkor a többiek segítségére lehetnek azokkal az ajánlásokkal, amiket a tutorial nem ad meg.

A Dungeons & Dragons-hoz viszonyítva a Roll20 rendelkezik egy beépített térképrajzoló résszel, aminek nagy része előfizetős tartalomként jelenik meg, de eltekintve a fizetős résztől, még így is sokat ki lehet hozni belőle anélkül, hogy egy harmadik alkalmazást kellene használni. Emellett megtalálhatóak különböző eszközök is a játékosok és játékmesterek segítségére, például név- vagy különböző térkép generátorok, stb.

A Roll20 egy remek tulajdonsága, mivel alapvetően virtuális platformként indult, hogy a játék folyamatainak bemutatására készítették a fejlesztők egy interaktív útmutatót, amely végig vezeti a felhasználót a játéktéren és a funkciókon, amikhez 1-2 perces magyarázó videókat is csatolnak. Itt már elérhetőek a karakterek tulajdonságai (karakterlapok), bár kicsit elrejtve a *Journal* menüben érhető el. Figyeltek arra a készítőik, hogy azon karaktereket a játékosok ne piszkálhassák, amiket nem ők hoztak létre. Ez azért is fontos, mivel, ha játékosként lehetne befolyásolni például az ellenfelek tulajdonságait, az igazságtalan lenne a játékmesterrel szemben, aki egy átvitt értelemben istenként van jelen. Ha pedig egy másik játékos karakterbe szerkesztene bele valaki, az a csapattársakkal szemben lenne igazságtalan. Érdekesnek tartom, hogy amikor a játékmester elindítja a játékot, a csatlakozott játékosokat a rendszer nem lépteti tovább automatikusan a játékfelületre, hanem maguknak kell „elindítani” a játékot. Maga a játékélmény, az előbb említett dolgokat figyelembe véve jobb, mint a D&D Beyond

rendszerében, de ez részemről elvárás egy olyan alkalmazástól, mely régebb óta van piacon.

1.3. Foundry VTT

A Dungeons & Dragons-szal és a Roll20-val ellentétben a Foundry VTT egy egyszeri alkalommal megvásárolható asztali alkalmazás. Az ára 43 euró, melybe nem tartozik bele az ÁFA, így magyarországi megvásárlás esetén körülbelül 55 euró. Ez a másikkal összehasonlítva viszonyítva sokkalta jobban megéri, mivel a D&D Beyond és a Roll20 is előfizetős rendszerek. Előbbi évi egyszeri fizetés esetén a legolcsóbb árazással is 33 euró, ami a funkcionális különbségeket tekintve nagyon túlárzott a Foundry-hoz képest, míg az utóbbi évi egyszeri fizetést tekintve a legolcsóbb árazással 60 euró, mely még mindig drágának tekinthető, mivel az az előfizetési osztály, mely felérhet a Foundry rendszerével 110 euróért érhető el.

Egyetlen hátránya a Foundry VTT-nek, hogy nem támogatja közvetlenül a saját térképek létrehozását, akárcsak a D&D Beyond. Azonban lehetőség van saját játékszervert hosztolására vagy harmadik fél által biztosított szervereket is igénybe lehet venni, ha nem lennénk túl magabiztosak a portforwarding terén. Ez az alkalmazás is rendelkezik beépített audió és videóhívási funkcióval. Viszont ahhoz, hogy ez használható is legyen, saját szerver hosztolás esetén engedélyezni kell SSL/HTTPS használatát a szerveren.

A játékmesternek lehetősége van egy játék létrehozása előtt hozzáadni a szoftverhez a közösség által készített egyéb plusz tartalmakat, mint egy kézre állóbb menedzsment interfész, 3D dobókockák és azokhoz tartozó kocka dobó tányérok (a vizuális ingerek fokozására), új jelzők és pályára lerakható elemek, stb. Ezután a játékkészítés felületén nevet ad a játéknak, kiválasztja a preferált rendszert, például a Dungeons & Dragons-t – ezen kívül még körülbelül 300 rendszer közül választhat – ezt a későbbiek során nem változtathatja meg. Megadhatja, hogy mikor lesz a következő játékalkalom és még egyéb dolgokat (világ leírás, csatlakozási oldal témája és háttere).

A játéktérben jobb oldalon található egy panel, melyben rengeteg vezérlési lehetőség van. Alapvető játékbeállítások, jelenet beállítások, jegyzetek, átvezetők, tárgyak, szereplők, hanghatások, harc menedzsment. Jelenetek között alapértelmezett módon csak játékmesternek lehetséges a léptetés, azaz csak a játékmester mozog át és vihet át egy játékost vagy akár az összeset is. Ez logikus, hiszen rendezőként nem

szerencsés, hogy egyes színészek az egyik színdarab közben elkallódnak és egy másikat kezdenek játszani. Természetesen lehetséges engedélyezni a játékosoknak is a navigálást a jelenetek között.

A játékosok csatlakozását jelszóval lehetséges korlátozni, hogy biztosan az csatlakozzon, akinek jogosultsága van hozzá. Csatlakozás után lehetőség van a játékosprofil személyre szabására, ahol a nevet és a játékos színét állíthatja be a felhasználó, ennek a megjelenítésnél van csak szerepe. Itt kiválaszthatja a karaktert, amit a játékmester rendelt hozzá, ha már létezik. Ha nem létezik, a játékmester létre tud hozni üres karakterlapokat, melyeket minden olyan játékoshoz hozzárendelhet, akinek nincsen karaktere. Emellett a játékmester előre elkészített karaktereket is hozzárendelhet a játékosokhoz.

2. Funkcionális specifikáció és rendszerterv

2.1. Képernyőtervek

Az alkalmazás képernyőterveit a Figma nevű webes szoftverrel készítettem el.

2.1.1. Főoldal



2.1. ábra. A főoldal képernyőterve.

A **2.1. képen** a főoldal terve látható. Ez az oldal az egyik legfontosabb része a webalkalmazásnak, mivel ezt látja először a felhasználó. A felület úgy lett megtervezve, hogy amennyire csak lehet tükrözze az alkalmazásban megtalálható funkciókat. Ezek egy része a középen lévő kártyákban van jelen, amik az oldalon való navigálást hivatottak gyorsítani, míg a részletesebb vezérlés a bal felső sarokban a navigációs menüben lehetséges. A jobb felső sarokban a felhasználóval kapcsolatos funkciók találhatóak (regisztráció és belépés, profil és kijelentkezés).

2.1.2. Statikus nézetoldalak

A *Tájak és Királyságok* oldala egy eredet történettel szolgál az *Ébredő Világok* szerepjáték világához. Ezen a felületen található a világ teremtésének a leírása, a kontinensen megtalálható területek és a mágia szerepének rövid ismertetése.

A világ oldal – **Melléklet A. ábra.** – a szerepjáték világának bemutatására szolgáló oldalakat tartalmazza. A navigáció ezen az oldalon már csakis a képernyő közepén található kártyákkal történik.

A fajok oldala – **Melléklet B. ábra** – funkcionális téren megegyezik a Világ oldallal. A hat kártya a hat fajcsoport alfajainak leírásához vezet.

Egy faj leírásának felülete az adott fajcsoportba tartozó alfajok listázására szolgál – **Melléklet C. ábra**. A tartalom nem statikusan kódolt, hanem dinamikusan generálódik Angular Routing segítségével egy előre definiált TypeScript objektum felhasználásával.

A térkép oldalon – **Melléklet D. ábra**. – található a világtérkép, amit én készítettem a Wonderdraft nevű szoftver segítségével (a képernyőterven még nem ez szerepel). A térkép a Leaflet könyvtárat használja. Itt a felhasználónak lehetősége van megvizsgálni a világ különleges helyszíneit, melyeket a térképen egyedi térképi jelölők mutatnak. Ezeket a bal oldali panelen lévő gombokkal lehet megjeleníteni és eltüntetni. Elérhető egy egyszerű keresési funkció is, ami a helyszínek nevében és leírásában keresi a beírt szövegrészletet függetlenül a nagy- és kisbetűktől. A helyszínek, hasonlóan a fajok-alfajokhoz, egy-egy TypeScript tömbben objektumokként vannak tárolva és beimportálva a komponensbe.

A rendszer oldal – **Melléklet E. ábra**. – kialakítása egy nyitott könyvet imitál. Az oldalak közötti léptetést a képernyő két szélén található gombbal végezheti a felhasználó. Az oldalak tartalma külön-külön komponensekben található. A fejezetek közötti navigációt a könyv jobb felső sarka mellett található könyvjelzők segítik, amelyek azonnali ugrást tesznek lehetővé a három fő témakör kezdőoldalára.

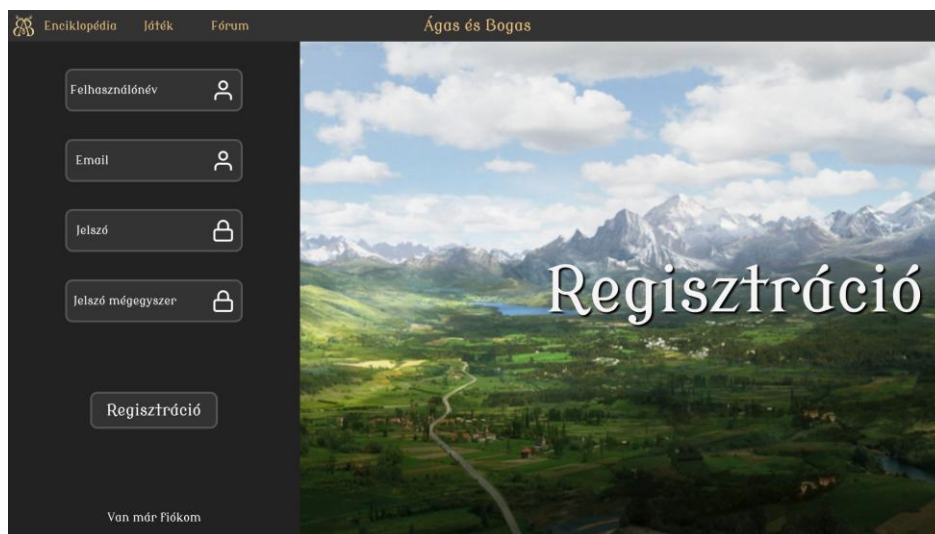
2.1.3. Bejelentkezés oldal



2.3. ábra. A bejelentkezés oldal képernyőterve.

A bejelentkezés oldalát a **2.3. kép** ábrázolja. A regisztrált felhasználók ezen az oldalon tudják azonosítani magukat. A bejelentkezéshez szükséges a felhasználó email címe és a jelszava, melyek beírása után a Bejelentkezés gombra kattintva az azonosítást a Firebase Authentication végzi el. Hibás adat esetén hibaüzenet jelenik meg, hogy tájékoztassa a hibáról a felhasználót. A bejelentkezési panel alján az „Elfelejtettem a jelszavam” a jelszó-visszaállító folyamatot indítja el, míg a „Nincs még fiókom” link átirányítja a vendég felhasználókat a regisztrációs felületre.

2.1.4. Regisztráció oldal



2.4. ábra. A regisztráció oldal képernyőterve.

A regisztrációs oldalt a **2.4. ábra** ábrázolja. A vendég felhasználóknak ezen az oldalon van lehetőségük regisztrálni az alkalmazásba. Egy felhasználó regisztrálásához a vendégnek szüksége van egy felhasználónévre, egy email címre és egy jelszóra, amit egyszer meg kell ismételnie az elgépelésből fakadó hibák elkerülése érdekében.

A regisztráció gombra kattintva a regisztrációs folyamat elindul. A rendszer először aszinkron módon ellenőrzi az adatbázisban a választott felhasználónév, majd az e-mail cím foglaltságát. Amennyiben az ellenőrzés sikeres, a Firebase Authentication segítségével létrejön az új fiók, majd a rendszer automatikusan be is lépteti az új felhasználót az alkalmazásba. Hibás adat, nem megfelelő formátum vagy már foglalt azonosító esetén a felület hibaüzenettel tájékoztatja a felhasználót. A panel alján a „Van már fiókom” link átirányítja a felhasználót a bejelentkezési felületre.

2.1.5. Karakter készítő oldal

The screenshot shows a character creation interface with a dark theme and gold accents. At the top, there's a navigation bar with 'Enciklopédia', 'Játék', ' Fórum', and 'Ágas és Bogas'. On the right of the bar are 'Név' and a user icon. The main area is divided into five panels. The 'Személyes' panel on the left has input fields for 'Név', 'Nép', 'Szerep', 'Népi tulajdonság', 'Otthon', and a larger 'Egyéb tulajdonság' field. The 'Tulajdonságok' panel in the top middle has a 3x3 grid of input fields for 'Erő', 'Ész', 'Életerő', 'Ügyesség', 'Fortély', 'Kifarás', 'Akarat', and 'Stressz'. Below it, the 'Erények és hátrányok' panel has a 'Erények' list box and a 'Hátrány' text input. The 'Tárgyak' panel to its right has a single '+' button. The 'Felszerelés' panel on the right has slots for 'Bal kéz', 'jobb kéz', and 'Páncél'.

2.5. ábra. A karakter készítő oldal képernyőterve.

A regisztrált felhasználók készíthetnek karaktereket, amiket játék során irányíthatnak. A felületet – **2.5. ábra** – öt külön álló részre lehet bontani.

A karakter személyes tulajdonságait a név kivételével, mely egyszerű szöveges beviteli mező, legördülő listából választja ki a felhasználó. A „Faj/Nép” kiválasztásával automatikusan kilistázódnak az adott fajhoz tartozó bónusz tulajdonságok, a fajok részen említett objektumokból, ezért a „Népi tulajdonság” és „Otthon” kiválasztása csak a Faj meghatározása után lehetséges. Ezen tulajdonságokból eredő egyéb bónuszokat az „Egyéb tulajdonságok” részen láthatja a felhasználó.

A tulajdonságok panel csak szám (numerikus) típusú beviteli mezőket tartalmaz. A fő tulajdonságok (Életerő/Hárítási pont és Stressz pont) értéke nem lehet negatív szám, míg a fizikai és mentális tulajdonságok esetében a felhasználó -3 és +3 közötti módosító értéket adhat meg.

Az erény és hátrányok, illetve a felszerelés és a tárgyak meghatározása is legördülő listával valósul meg. Ezek az értékek megadása az előző kettővel ellentétben opcionális.

2.1.6. Kaland készítő oldal



2.6. ábra. A kaland készítő oldal képernyőterve.

A kalandkészítés felülete – **2.6. ábra** – két fő egységből áll, az események és az NPC-k (Non-Player Character) meghatározása. A közepen megjelenő térkép azonos a térkép oldalon lévővel. Ennek az esemény létrehozásában van szerepe a helyszín kiválasztása szempontjából.

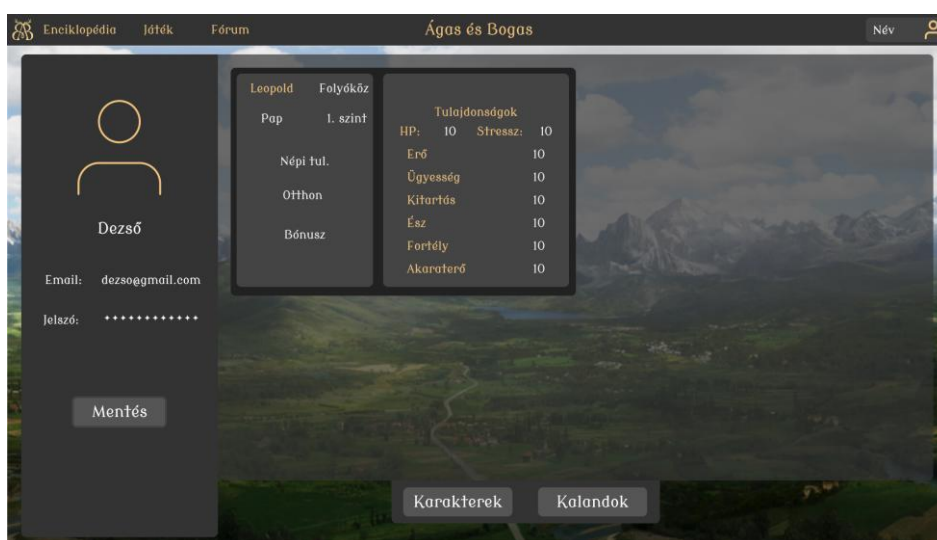
Esemény létrehozása az esemény panelen megjelenő hozzáadás gombbal kezdeményezhető. Ekkor megjelenik egy új panel, melyen az esemény neve, egyszerű szöveges bemenetként, a helyszín, egy legördülő listában vagy a térképen kiválasztva, illetve leírás és történet adható meg, szövegdobozokban. A létrehozás gombra kattintva létrejön az esemény objektum, mely belekerül egy tömbbe, ami majd a kaland létrehozásakor rendelődik a kalandhoz. Miután létrejött az esemény, lehetőség van annak szerkesztésére és törlésére. Előbbinél megjelenik a létrehozás felülete, betöltve a szerkesztendő esemény adatait, melyek megváltoztatása és mentése után azok felülírják az előzőeket.

NPC hozzáadása az eseményhez csak akkor lehetséges, ha a felhasználó kiválasztott egy eseményt. Ha ez megtörtént, megjelenik a hozzáadás gomb, melyre kattintva egy új panel jelenik meg. Itt megadható az NPC neve szöveges bemenetben, a hozzáállás rádiógombokkal választható, mely azt határozza meg, hogy neutrális vagy ellenséges. Az NPC játék eleji megjelenítését és a kereskedői szerepet jelölőnégyzettel lehet állítani. Utóbbi bepipálása felhoz egy felületet, mely kereskedelemre alkalmas tárgyakat tartalmaz. Mindegyikhez tartozik egy jelölőnégyzet (elérhető) és egy

numerikus bemeneti mező (darabszám meghatározása). Ezek beállítása után az NPC-hez tartozó karakter hozzáadása következik. Egy legördülő menüből lehet kiválasztani egy szimpatikus karaktert, melyet a felhasználó korábban készített, vagy az „Új karakter” rész lenyitásával teljesen egyedi készítésű új karakter adható hozzá.

2.1.7 Profil oldal

A profil felülete hivatott a felhasználó által készített karaktereket és kalandokat listázni – **2.7. ábra**. A két téma között a konténer alatt található gombok segítségével történik a váltás. A panel bal oldalán jelenik meg a felhasználói fiókhoz tartozó felhasználónév és email cím.

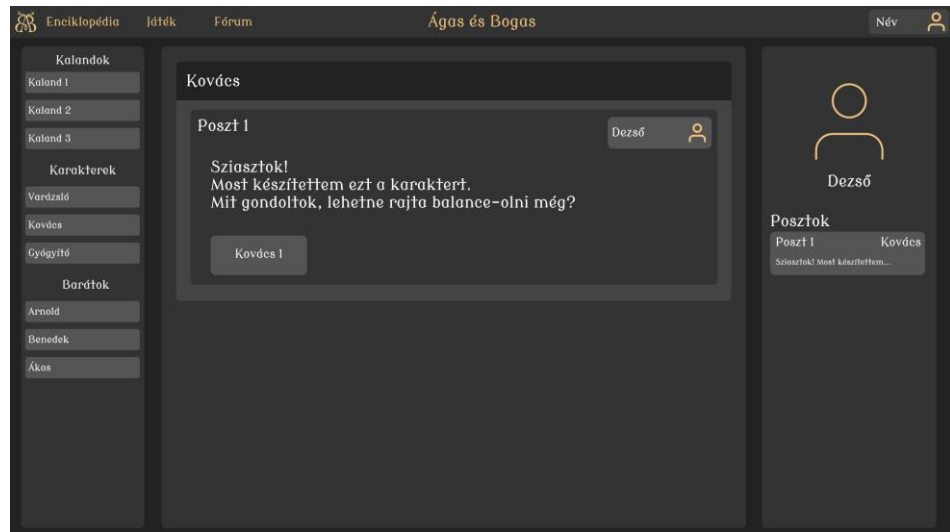


2.7. ábra. A profil oldal képernyőterve.

A megjelenített karakterek egy kártyát imitálnak. A kártya bal oldalán találhatóak a karakter személyes tulajdonságai. A Népi tul. és Otthon gombok, melyekre kattintva az adott tulajdonság leírása jelenik meg. A jobb oldalon a fő, fizikai és mentális tulajdonságok találhatóak.

A listázott kalandok megjelenítése eltér a karakterekétől. Ez kitölti szélességében a konténert. Megjelenik a kaland neve és az események száma a kalandban. Kattintással lenyitható, itt jelennek meg az események egymás mellett. Egy eseményen, ha nem lett csatolva NPC hozzá, megjelenik a név, a leírás és a hozzá írt történet míg, ha lett NPC csatolva, akkor egy NPC gomb is megjelenik, mely egy kisebb panelen listázza ki az adott esemény NPC-it.

2.1.8. Fórum oldal



2.8. ábra. A fórum oldal képernyőterve.

A fórum felülete – **2.8. ábra** – három részre tagolódik. A bal oldali panel a böngészhető fórumokat listázza ki. Középen a fórum tartalma jelenik meg. A jobb oldalon található panelen a bejelentkezett felhasználó neve és posztjainak listája található.

Poszt létrehozása csak bejelentkezés után lehetséges. Ekkor a felhasználó posztjai felett megjelenik egy gomb „Új poszt+” felirattal. Rákattintva egy új panel jelenik meg. Itt a felhasználó kiválaszthatja egy legördülő menüben, hogy melyik fórumra szeretne posztolni. Üres poszt létrehozása nem engedélyezett, ezért kell a posztnak egy cím és egy szöveg is. Fórumtól függően a felhasználó csatolhat karaktert vagy kalandot a poszthoz. Ezek csatolmányként megtekinthetők a poszt alján.

2.1.9. A játéktér

Az alkalmazás játék funkciója három fő részre osztható. Magára a játéktérre, a játék várostermőjára és egy menüre, ahol a felhasználók létrehozhatnak vagy elindíthatnak egy játékot, vagy csatlakozhatnak egyhez. Ezt két panelen végezhetik.

Az első panelen a játék létrehozása történik. A játéknak meg kell adni a nevét, egy szöveges bemenetben, ezután egy legördülő listában ki kell választani a játszani kívánt kalandot. A kaland kiválasztása után egy szám típusú bemenetben meg kell adni a maximális játékos számát 1-6-ig, majd egy jelölőnégyzetben beállítani a játék publikusságát. A létrehozott játék a panel melletti konténerben jelenik meg, ahol a neve, a játékoskorlát és publikus elérhetősége is látható. A játék elindítását ezen listaelemen található indítás gombbal lehet elvégezni. Megtalálható még egy törlő gomb is, mellyel a játékot lehet törölni. Játékindítás után a játék várótermébe kerül a felhasználó és megkapja a játékmesteri rangot.

Játékhoz csatlakozni a második panelen lehetséges. A csatlakozáshoz, ha privát a játék, szükség van egy csatlakozási kódra, ezt a játékmester biztosítja a játék elindítása után, és egy karakterre, amit egy legördülő menüből lehet kiválasztani. Csatlakozás után a játék várótermébe kerül a felhasználó és megkapja a játékos rangot.

A váróteremben a játékosok szemrevételezhetik egymás karakterét, a kalandot és annak eseményeit, jelezhetik felkészültségüket a játékra és menedzselhetik a hanghívást. A játékmester ezeken felül még a játékosok kirúgására is képes. Ha minden játékos készen áll, a játékmester elindíthatja a játékot, ekkor minden felhasználó a játéktérre navigál automatikusan.

A játéktér felülete – **2.9 ábra** – több panelből áll. Játékos és játékmester különböző vezérlésekkel rendelkezik, de vannak azonosak is. Ide tartozik az esemény panel, ahol az éppen soron lévő esemény leírása található egy térképpel, mely a helyszínt mutatja, a játékosok és az eseményhez tartozó NPC-k listája és az egyéb vezérlés panel.



2.9. ábra. A játéktér képernyőterve.

A játékos akciói a játékosok listája alatti panelen helyezkednek el. Az akciók gombjain az akciót szimbolizáló ikon és tooltip segíti a játékosokat. A játékos karakterének hátizsákja és az abban található tárgyak a felület felső felében, a játékoslista mellett helyezkednek el. Ez három ablakra van osztva, melyek között a panel fejlécében található gombokkal lehetséges a léptetés. A hátizsák alatt foglal helyet a karakter panele. Itt megjelennek a karakter tulajdonságai, melyekre kattintva a játék során egy próba dobása történik 3D kockákkal.

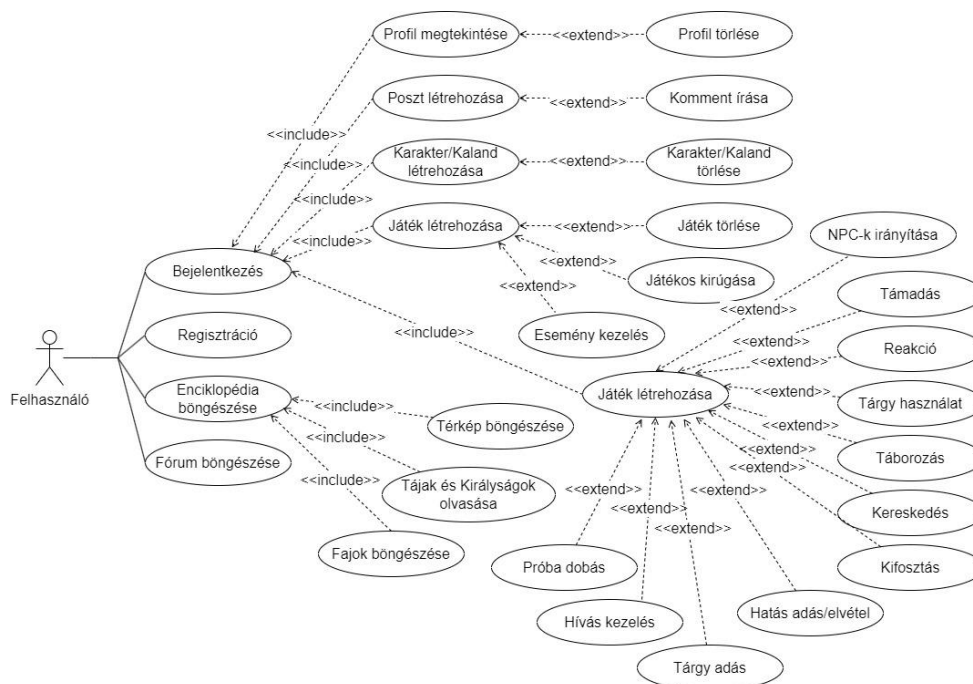
A játékmester egyedi felületéhez tartozik a játékoskontroll panel, mely a játékoslista mellett teljes képernyő magasságban látható. Ez a panel három részre van osztva, felső, középső és alsó szegmensre. Felül az éppen soron lévő játékos elvégzett akciói találhatóak.

Középen a játékmester megtekintheti a játékos tárgyait, azokhoz hozzáadhat újakat. Ezt egy új felületen egy legördülő menüből egy már létező tárgyat választva teheti vagy egy teljesen új tárgy létrehozását követően adhatja azt hozzá. Az új tárgynak meg kell adni a nevét és leírását, ki kell választani a típusát és megszabni a használatának számát (legalább 1). Egyes tárgyak eredményezhetnek valamilyen hatást, ezt jelölőnégyzettel lehet szabályozni. Hatás hozzáadása a tárgyhoz egy új panelen történik, ahol a hatás típusát kell kiválasztani, megadni a hatás értékét, időtartamát és a hatás célpontját.

Az alsó szegmensben a játékos karakteréhez lehet hozzáadni új hatásokat vagy a meglévőket eltávolítani. Új hatás hozzáadása egy új felületet jelenít meg, ahol a játékmester kiválaszthatja a hatás típusát, módosító értékét és időtartamát.

A játékmester elérheti azt a vezérlési felületet is, amit a játékosok használnak. Ezt úgy teheti meg, hogy kiválaszt egy NPC-t az eseményhez tartozó listából. A listában a kiválasztás mellett az NPC-k láthatósága is állítható, ennek eredménye, hogy a játékosok felületén csak azok az NPC-k láthatóak, amiket a játékmester láthatónak jelöl.

2.2. Szereplők és funkcióik



2.10. ábra. A webalkalmazás Use Case diagramja.

Az alkalmazásban felhasználói szempontból 4 szerepkör különböztethető meg, ebből kettő az oldal böngészésekor él, míg a másik kettő a játéktérben vannak jelen.

A böngészéskor életben lévő szerepkörök az egyszerű látogató és a regisztrált felhasználó. A látogató a hierarchia legalsó fokán áll, kevés jogosultsággal rendelkezik, csak a szerepjáték szabályzatának olvasása, a fórum böngészése szigorúan olvasói szerepből, illetve a regisztráció és a bejelentkezés (ha már regisztrált) tartozik hozzá. Ezzel szemben egy regisztrált és bejelentkezett felhasználó eléri az összes lehetőséget, amit az alkalmazás kínál (karakter/kalandkészítés, játéktér elérése, posztolás).

A játéktér csak az azonosított felhasználók érhetik el. A felhasználók a játékokat itt hozhatják létre vagy csatlakozhatnak egyhez. A választásuktól függően felvehetik a játékmester vagy a játékos szerepét. A két szerepkör közt vannak a kettőt átfedő és egyéni funkciók is. Például a játékmester kiváltsága a játékosok kirugása, az esemény kezelés, hatás/tárgyak adása vagy az NPC-k irányítása, vagy a játékosok

kereskedés, kifosztás és táborozás funkciói. Közös funkciókba tartozik például a támadás és reakció, hívás kezelés és a próba dobás. A játéktér hierarchiáját tekintve a játékmester magasabban áll, ezért több egyedi funkcióval rendelkezik, ami egy kaland lebonyolításához szükséges.

3. Használt technológiák

3.1. Backend

Az alkalmazás valós idejű kommunikációt biztosít a szerver és a kliensek között. A projekt a Backend-as-a-Service (BaaS) modellt alkalmazza, amihez a legmegbízhatóbb és a projekt igényeinek leginkább megfelelő választás a Firebase volt.

A rendszer adatainak tárolására a Firestore adatbázist vettem igénybe. Ez egy dokumentum alapú NoSQL felhőadatbázis, ami közvetlenül kezeli a valós idejű adatfolyamokat, szemben a relációs adatbázisok REST API lekérdezéseivel. Ez elengedhetetlen a platform játékfelületének működése szempontjából. A Firestore a háttérben nyitott kapcsolatokon keresztül automatikusan szinkronizálja a kliensek között a játék közben megváltozott adatot, például harc során a sebzés kiosztásakor, így biztosítva a gördülékeny játékmenetet[1].

Az felhasználók azonosítását és a játékmenet kezelésének egyes részeit a Firebase Authentication végzi, ami garantálja a biztonságos bejelentkezési folyamatot és támogatja a regisztrált felhasználók kezelését e-mail és jelszó alapú hitelesítéssel, titkosított tokenek használatával, hogy a jelszavak tárolása ne az adatbázisban történjen[2]. Mivel az alkalmazás elsősorban frontend alkalmazás, ezért fokozott hangsúly van a biztonságon. Ezt a Firestore Security Rules biztosítja. Ezek szerveroldali szabályok, melyek megszabják, a felhasználói kérések alapján, hogy egyes műveletek hogyan vagy egyáltalán végbe mehetnek-e[3]. Például a fórumot böngészve csak akkor lehet kitörölni egy posztot, ha a törlés kérését végrehajtó felhasználó azonosítója megegyezik a poszt létrehozójának azonosítójával.

Az alkalmazás publikálásához a Firebase Hosting szolgáltatást használom. Ez a szolgáltatás SSD-alapú tárhelyet, CDN-t és SSL/TLS (HTTPS) tanúsítványt is biztosít[4].

3.2. Frontend

Frontend oldalon az alkalmazás az Angular keretrendszert használja. A piacon számos modern UI könyvtár elérhető, mint a React vagy Vue.js. Azonban egy virtuális szerepjáték platform komplex felépítést igényel. Ezt az Angular komponensek modularitása és a TypeScript nyelv típusossága tökéletesen kielégíti[6]. Mindemellett a Firebase rendszerével tökéletes kompatibilitása, a `@angular/fire` könyvtár terén, volt a legerősebb érv a használata mellett.

A fejlesztés során az MVC/MVVM[5] mintáját követve sok olyan komponenszt hoztam létre, melyek dinamikusan tölthetők fel adatokkal. Például a főoldal navigációs kártyái, amik a Világ és a Fajok menü oldalán is megjelennek, vagy az alfajok leírását tartalmazó oldal. Mivel a keretrendszer a TypeScript nyelvre épül, tökéletes választás volt számomra az évek során elsajátított JavaScript tudásom kamatoztatására. Emellett a szerepjáték rengeteg specifikus adatstruktúrával rendelkezik (különböző tárgyak, szereplők, állapotmódosítók), amihez a TypeScript interfészeket és enumokat használtam a futási időben keletkező hibák kiszűréséhez.

A grafikus felhasználói felület kialakításához felhasználtam az Angular Material UI komponenskönyvtárat. A könyvtár használatának oka az volt, hogy rengeteg letisztult dizájnelemmel rendelkezik, melyek közül némelyiknek több változata is elérhető (pl. Material Button). A GUI másik fontos építő eleme a Reactive Forms, ami a komplexebb űrlapok kitöltésének vezérlését szolgálja. Segítségével a szerepjáték bonyolultabb adatstruktúráinak, mint a karakterek tulajdonságai (a fő, a fizikai és a mentális), és a dinamikus validációs szabályoknak a kezelése egyaránt végezhető TypeScriptből.

Az Angular legerősebb pontja a reaktív programozás lehetősége (Reactive Extensions for JavaScript, röviden RxJS). Az RxJS segítségével az adatbázisból érkező aszinkron adatfolyamokra (Observable-ként) fel lehet iratkozni és a bejövő adatot különböző operátorokkal (*pipe*, *take*, *map*, *filter*, vagy *switchMap*) még az előtt feldolgozni, mielőtt az a felhasználóhoz kerülne[7].

A világtérkép elhelyezéséhez a weboldalon a Leaflet könyvtárat választottam. Ez egy nyílt forráskódú, lightweight könyvtár, ami az általam rajzolt térkép kiegészítését úgy segíti, hogy különböző rétegeket rendel a képhez, amire egyedi jelölőket/ikonokat és formákat rajzolhatok (pl. a vasút rengeteg koordináta összekötésével alkotott vonal)[8].

3.3. Webes audiokommunikáció

Szerepjáték során az élmény elengedhetetlen része a játékmester és a játékosok közötti kommunikáció. A játékosok használhatnak külső szoftvereket (Discord vagy Skype), de a projekt célja egy „All-in-one” platform létrehozása volt, így ezt a funkciót a böngészőkbe beépített WebRTC technológia segítségével készítettem el.

A WebRTC egy API szabvány, amely lehetővé teszi a böngészők közötti peer-to-peer valós idejű hang- és videó adatkapcsolatokat[9] bármilyen harmadik féltől származó megoldás használata nélkül, ezzel egy gyors, alacsony késleltetésű rendszert alkotva.

A peer-to-peer kapcsolatok kiépítése az úgynevezett „Signalling” folyamattal kezdődik. E folyamat során a két böngésző a tűzfalon és a NAT hálózatokon átjutva, a csatlakozáshoz szükséges metaadatokat (hálózati portok, IP címek) cserél egymással[10]. Ehhez a folyamathoz szükség van egy Signalling szerverre. A szerver létrehozásának elkerülése érdekében a PeerJS könyvtárat használtam, ami egy absztrakciós réteget biztosít a WebRTC API fölé, leegyszerűsítve a kapcsolatok felépítését a játékosok között[11].

4. Az alkalmazás magas szintű felépítése

Az *Ébredő Világok* platform egy elosztott, felhő alapú architektúrára épül. A rendszer három különálló futtatási környezetre bontható: kliens, backend és egy külső szolgáltatási PeerJS környezet.

4.1. Kliens környezet

Mivel a platform egy egyoldalas webalkalmazás Angular keretrendszerben, ezért a felhasználó gépe végzi a nagyobb munkát. Az oldal első megnyitásakor a kliens letölti a szerverről a gyökérfájlokat és ezekkel dolgozik az ablak bezárásáig. A környezetet a kliensgép böngészője futtatja, ahol a JavaScript motor értelmezi és végrehajtja az Angular kódot.

Ez a környezet felel a grafikus réteg megjelenítéséért, amit a böngésző renderelő motorja rajzol ki. Itt fut a világtérkép megjelenítésére használt Leaflet könyvtár is. A rendszer eléri a kliens számítógép memóriáját és a videokártyát, hogy az erőforrást felhasználhassa a térkép nagyításának és mozgatásának kiszámításához.

Az állapotok nyilvántartását is ez a környezet végzi. Szerepjáték során ez lehet különböző karakterek életerejének változása, akciók végrehajtása, stb. Ezek a kliens memóriájában futnak le az Angular szolgáltatások (Angular Service) és RxJS segítségével. Például harc közben, mikor egy megtámadott játékos nem reagál a támadásra, akkor a kliens hajtja végre a sebzésének kiszámítását és végeredményt, végállapotot küldi el a szervernek.

A játék során használt hanghívási folyamatokat is a kliens végzi a böngésző beépített API-jain keresztül. Jelen esetben a Web Audio API[12] és a MediaDevices API[13] használatával a kliens hozzáfér a számítógép mikrofonjához és hangszórójához. Ezeket a perifériákat összeköti a kliens a WebRTC protokollal, hogy létrejöhessen a peer-to-peer kapcsolat a többi játékos között.

4.2. PeerJS Signalling környezet

A valós idejű kommunikáció működéséhez szükség van egy közvetítő környezetre, ez a jelzőszerver a peer-to-peer kapcsolatok létrehozásához, amit a PeerJS API-ja valósít meg.

Két böngésző alapvetően nem képes csatlakozni egymáshoz, mert a két klienst routerek, tűzfalak védik és NAT rejti el. Ezért van szükség a PeerJS szerverkörnyezetére, ami csomópontként viselkedik. A célja, hogy a kapcsolat létrehozásához szükséges metaadatokat (publikus IP-ket és nyitott portokat tartalmaznak) kicserélje a kliensek között. A játéktérhez csatlakozáskor a PeerJS szerver generál egy azonosítót, aminek a segítségével a szerveren keresztül a kliensek megtalálják egymást és elkezdhetik felépíteni kapcsolatot SDP (Session Description Protocol) és ICE (Interactive Connectivity Establishment) segítségével[10].

A környezet sajátossága, hogy csak átmeneti ideig él kapcsolat a klienssel. Ez azt jelenti, hogy miután megtörténik a kézfogás a hívó felek között és a kapcsolat teljesen felépült, a szerver kilép a kommunikációból. Ezt követően a hívás egy hálós (mesh) topológiában működik, közvetlenül a kliensek között (peer-to-peer). A WebRTC szabványból kifolyólag hangátvitel titkosított a végpontok között (End-to-End – DTLS/SRTP protokoll)[9].

4.3. Backend környezet

Mivel az alkalmazás Backend-as-a-Service modellt használ, nevezhetjük „Serverless”-nek is. Ez azt jelenti, hogy a Google Firebase szerverei által biztosított funkciókat személyre szabva használja, és nem saját rendszert kell építeni az abszolút nulláról.

Az adatbázis a Firestore, ami egy kollekció-dokumentum alapú adatbázis (NoSQL), amit kifejezetten a valós idejű adatáramlásra optimalizáltak, így ez volt a fő oka a kiválasztásának. Ez úgy valósul meg, hogy a háttérben a kliensek és a szerver között nyitott a kommunikáció (kliensek feliratkoztak egy adatfolyamra). Amikor az adatbázisban változás történik az adott adatfolyamon, akkor a szerver újra elküldi azt klienseknek.

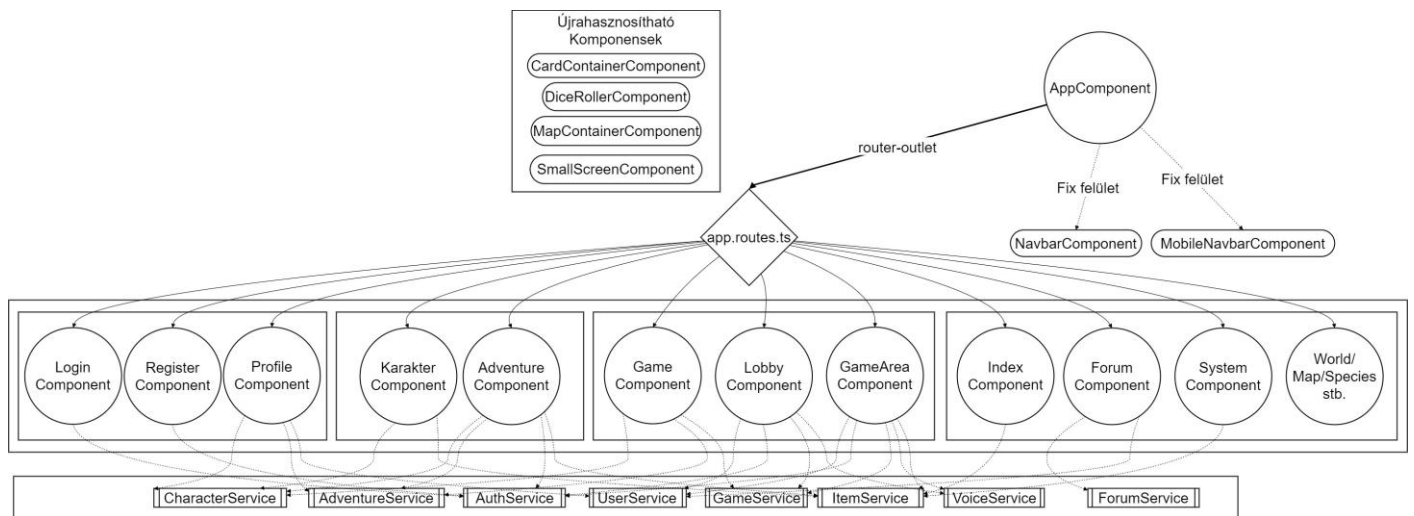
A felhasználók identitását a Firebase hitelesítő szolgáltatása ellenőrzi. A jelszavak nem a központi adatbázisban tárolódnak, így fejlesztőként nem kell a titkosításukkal (sózásukkal és hashelésükkel) foglalkozni. Sikeres azonosítás után a rendszer egy JSON Web Token-t (ezt a szerver kriptográfiával aláírja) állít ki a kliensnek, amivel a felhasználó a többi szolgáltatás igénybevétele előtt tudja magát igazolni.

Mivel a kliensből jönnek a kérések és az egy nem megbízható környezet, a kérések validálását a szerver végzi a Firebase Security Rules-on keresztül még az adatcsere előtt. Minden egyes adatbázis íráskor és olvasáskor a rendszer végig megy ezen a szabálylistán és leellenőrzi a kérelmező tokenjét, ha jogosultságilag és tartalmilag is megfelel a szabályoknak, akkor engedélyezi az adatok tranzakcióját.

A felhasználói fiók és a fórum posztjainak törlését nem a felhasználó végzi. Ez egy esemény vezérelt folyamat, ami triggerek hatására megy végbe, ezt Cloud Functions-nek nevezik. A törlésgombra kattintva elsül egy esemény a felhasználó törlésére, amit a szerver figyel és dolgoz fel, majd kitörli a felhasználót az adatbázisból, illetve az összes többi hozzá tartozó karaktert, kalandot és játékot is[18].

A weboldal hosztolását a Firebase Hosting végzi. Ez a környezet az alkalmazás statikus fájljait a világon több helyen tárolja (CDN), hogy a felhasználók minél alacsonyabb válaszidővel kapják meg a csomagokat a hozzájuk földrajzilag legközelebb lévő helyről, ezzel javítva a felhasználói élményt.

5. Modulok és komponensek



5.1. ábra. Az alkalmazás komponenseinek és moduljainak diagramja

Az alkalmazás 19-es verziójú Angulart használ, ahol már Standalone komponensek – **5.1. ábra** – vannak, szemben a hagyományos modulokkal (NgModule), ezért a rendszer belső felépítése logikai mappákból áll, amik követik a felelősségi körök szigorú szétválasztásának elvét (Separation of Concerns)[5].

5.1. A logikai rétegek

Az alkalmazásnak három fő logikai rétege van elválasztva a felelősségi körök alapján. Ezek a különböző oldalakhoz tartozó nézetek. Szintén grafikai és megjelenítési funkciókat látnak el az újrahasznosítható komponensek (pl. térkép), míg a fő logikát a szolgáltatások biztosítják.

5.1.1. Nézetek

A nézet komponensei, amelyek közvetlenül egy-egy útvonalhoz vannak rendelve, jelentik az alkalmazás gerincét. Ezeket a mappastruktúrában a „pages” mappa tartalmazza. Ide tartoznak a képernyő terveknél leírt oldalak. Mivel a nézetek fele (az enciklopédia részen) buta komponens ezek az **5.1-es ábrán** nem látszódnak, viszont az okos komponensek igen, mivel fontos műveleteket hajtanak végre adatokkal (szolgáltatások segítségével).

Az okos komponensek közül három kiemelhetően komplex oldal van, ezek a kaland- és karakter készítő komponensek és a játéktér megjelenítéséért felelős

komponens. Ezeket a modulokat javarészt űrlapok alkotják, amelyekből a kinyert adatokat a hozzájuk kapcsolódó Angular szolgáltatások dolgoznak fel, és végül továbbítanak..

5.1.2. Újrahasznosítható UI komponensek

A többszörösen felhasznált komponensek a nézeteket tartalmazó mappával egy szinten található „shared” mappában vannak. Ezek feladata a több komponensen átívelő megjeleníthetőség kód duplikálása nélkül. Természetesen jelen vannak több komponensben ismétlődő megjelenítési feladatot ellátó alkomponensek (vagy template komponensek), például egyedi listaelem megjelenítésére szolgáló komponensek a térkép oldalon, de ezek máshol nincsenek hasznosítva.

A legtöbbet használt ilyen újrahasznosított komponens a felső- és a mobilos nézethez tartozó navigációs menü komponensek, melyek minden oldalon megtalálhatóak szemben a térkép és kártya tartó komponensekkel. Előbbi az alkalmazásban a kalandkészítéskor, a játéktérben és a világtérkép oldalán van hasznosítva nagyjából azonos módon. Az utóbbi a fő-, a világ és a fajok oldalon van jelen.

5.1.3. Szolgáltatások

Az alkalmazás hálózati kommunikációját és az adatok állapotkezelését Angular szolgáltatások végzik. Ezek a megosztott erőforrások mappájában („shared”) találhatóak egy „service” almappában. Ezek a TypeScript fájlok Singleton-ként léteznek az alkalmazás szerkezetében, azaz csak egy példányban, egyetlen közös adatfolyamot biztosítva a komponensek részére[5].

A legfontosabb szolgáltatás az AuthService, ami a felhasználó hitelesítéseit végzi. Ezt a szolgáltatást szinte minden másik adatbázis műveletet végrehajtó szolgáltatás használja a felhasználói ellenőrzésekhez.

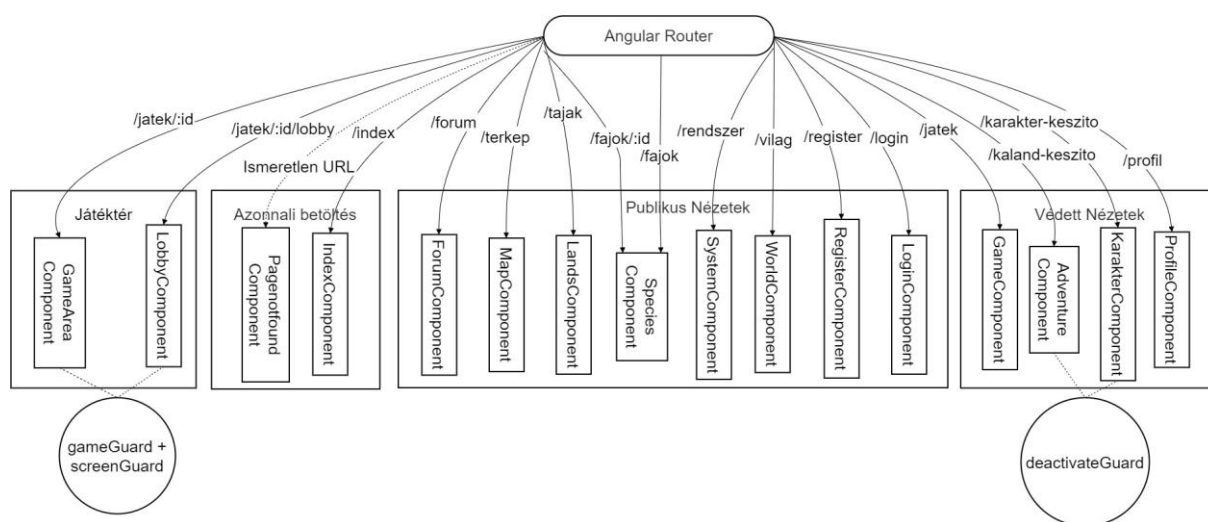
Az adatbázis műveleteket végző szolgáltatások (adventure, character, forum, game, item) egymáshoz nagyon hasonló struktúrát követnek. Mindben jelen van a listázás, a hozzáadás és törlés funkció, de mégis a minimális különbségek miatt a közös összevonásukra nem tudtam elég jó módszert kitalálni.

A legkomplexebb szolgáltatás a PeerJS hanghívását menedzselő szolgáltatás. Ezt csak az alkalmazás játék funkciója használja. Ez úgy valósul meg, hogy játékhoz

csatlakozáskor a felhasználónak engedélyezni kell a mikrofon használatát a böngésző részéről. Ha engedélyezve lett, akkor a szolgáltatás elkezd kiépíteni a kapcsolatot a felhasználó ID-jével.

5.2. Szoftveres kapcsolódási pontok

5.2.1. Strukturális kapcsolatok és Routing



5.2. ábra. Az alkalmazás navigációs diagramja

A komponensek közötti navigálás szabályait az *app.routes.ts* – **5.2. ábra** – fájl tartalmazza, ami lecseréli a hagyományos modul-alapú navigációt és köti össze a felhasználó által beírt URL-t a megfelelő komponenssel.

Fontos, hogy bizonyos oldalakat ne érhesen el egy látogató, példának okáért a karakterkészítés oldalát, ahova balszerencsés lenne eljutni úgy, hogy a hosszas idő alatt elkészített karaktert nem lehet elmenteni és utána használni. Ezen feladatot látják el a „guard”-ok, amik felügyelik a navigációs műveleteket még az alkalmazás gyökérszintjén és engedélyezik vagy éppen elutasítják a tovább lépést bizonyos szabályok alapján[14].

5.2.2. Függőségek injektálása (Dependency Injection – DI)

Az okos komponenseknek az alkalmazásban a GUI és a service-ek között valahogy kapcsolatot kell létesíteni. Ez nem úgy történik, hogy példányosítanak egy-egy osztályt minden szolgáltatásból, hanem az adott komponens a konstruktorán keresztül injektálja a szükséges szolgáltatásokat[15].

Ilyen például a *Firestore* az adatbázisból való lekérdezések felhasználásához[1], vagy a *CharacterService* a karakterkészítés komponensében a karakterek elmentéséhez, vagy a *ForumService* poszt létrehozásához, vagy a *VoiceService* a hanghívás kezeléséhez. Ez a függőség injektálás egyfajta laza csatolást biztosít a vizuális felület és az adatbázis-logika között, hogy ezen kettő egymástól elválasztva legyen, illetve könnyebben lehessen tesztelni a kódot.

5.2.3 Szülő-gyermek komponens interakció

Angular-ban két komponens közötti adatáramlást a komponens interakció végzi. Ez úgy működik, hogy a gyermek komponensekben az Angular `@Input`[16] direktívát használva lehet létrehozni egy reprezentáns változót. Ebbe a változóba a szülő komponensében lehet behejtesíteni a kívánt elemet. Ez jelenti a szülő felőli adatáramlást.

A gyermek komponensből a szülő felé adatot küldeni a `@Output`[17] direktívával lehet. Ezek esemény kibocsátókként (EventEmitter) vannak jelen, amiket hozzá lehet kötni különböző funkciókhoz (pl. egy gomb lenyomásához).

Az alkalmazásban egy példa ilyen szülő-gyermek komponens interakcióra a főoldalon (is) megtalálható kártya konténer, aminek bemenetül (`@Input`) egy TypeScript tömb adódik – **5.3. kódrészlet** – amikből előállnak a kártyák:

```
1 cards = [  
2   {  
3     name: 'Világ',  
4     img: 'assets/img/fooldal/vilag.webp',  
5     id: 'vilag'  
6   },  
7   {  
8     name: 'Játék',  
9     img: 'assets/img/fooldal/jatek.webp',  
10    id: 'jatek'  
11  },  
12  {  
13    name: 'Rendszer',  
14    img: 'assets/img/fooldal/rendszer.webp',  
15    id: 'rendszer',  
16  },  
17 ];
```

5.3. kódrészlet. Kártyastruktúra TypeScript tömbben.

A kártyákra kattintva lefut egy függvény, ami az adott kártya id-jét küldi fel a szülőnek. A szülőben a gyerekkomponens kimeneti jelzésének (*@Output*) fogadásához kapcsolt függvényben a kiküldött id-t használva a szülő komponens az injektált *Router* segítségével átnavigál az adott oldal komponensére.

6. Adatmodell

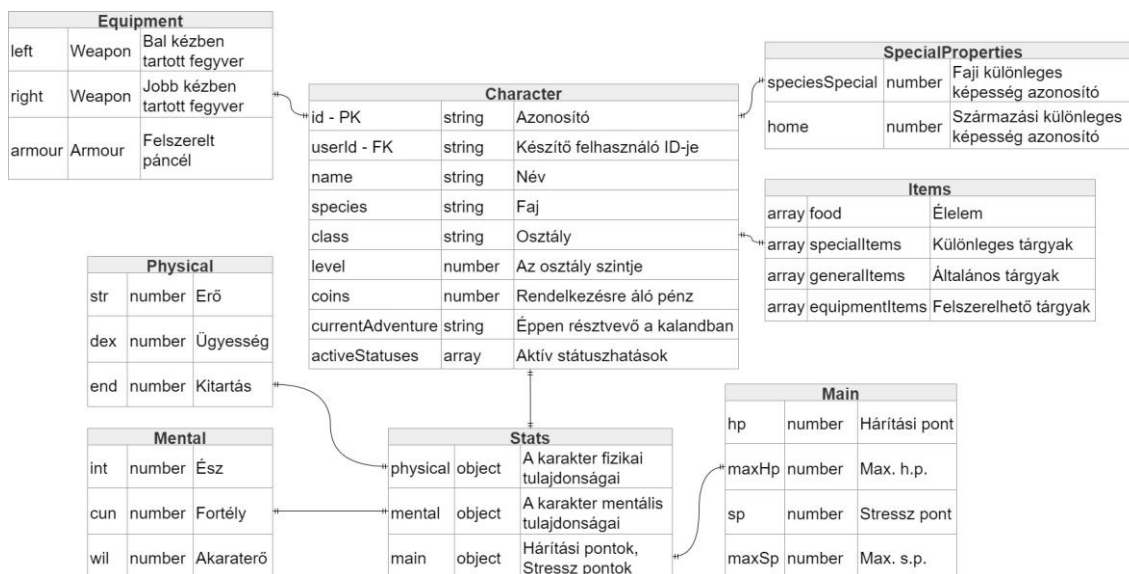
6.1. Felhasználó

User		
<i>id</i>	string	A felhasználó ID-je
<i>username</i>	string	A felhasználó neve
<i>email</i>	string	A felhasználó email címe
<i>characters</i>	array<string>	A felhasználó által készített karakterek azonosítói
<i>adventures</i>	array<string>	A felhasználó által készített kalandok azonosítói
<i>inGame</i>	null string	A felhasználó játékban van (ha igen – játék ID-je)

6.1. táblázat. A felhasználó adatszerkezete

Felhasználó a **6.1. táblázatban** szereplő adatszerkezettel jön létre regisztrációkor. Mivel a Firebase Authentication szolgáltatását használom, ezért nincs szükség jelszó eltárolására (a Google szerverein tárolódik és megy végbe az ellenőrzése). A felhasználóhoz tartozó karakterek és kalandok azonosítói egy-egy tömbben tárolódnak

6.2. Karakter



6.2. ábra. A karakter adatszerkezetének diagramja.

Az egyik legösszetettebb adatszerkezet a karakterekhez – **6.2. ábra** – tartozik. A logikailag elkülöníthető tulajdonságok külön objektumokat alkotnak az adatszerkezetben belül (specialProperties, equipment, stb.). Mivel az alkalmazás real-time működésű, ezért ez egy valós reprezentációja a játékban szereplő karaktereknek. A harci mechanika működésének egyszerűsítéséért külön választottam a fegyvereket a tárgytól. Megjelenik a *userId* mező, annak ellenére, hogy a *User*-nél tárolódnak az azonosítók, ez a Security Rules-ban lévő törlés előtti ellenőrzéshez szükséges (hasonlóan az Adventure-nél is).

6.3. Kaland



6.3. ábra. A kaland adatszerkezetének diagramja.

A játék egyik építőköve a kaland, ami a ránézésre nem tűnik nagy adatszerkezetnek – **6.3. táblázat** – ellentétben a karakterével, pedig mind a kettő fontos szereppel rendelkezik az alkalmazás működésében. A kaland fő egységét az események alkotják. Ezek olyan objektumok, amikhez tartozik egy helyszín, egy történet, egy leírás és egy név. Egy esemény létrehozása után lehetősége van a felhasználónak NPC-eket – **Melléklet Y. táblázat.** – hozzáadni, amikkel majd a játékosok – **Melléklet X. táblázat.** – találkozhatnak.

6.4. Fórum

6.4.1. Fórum felhasználó, poszt és komment

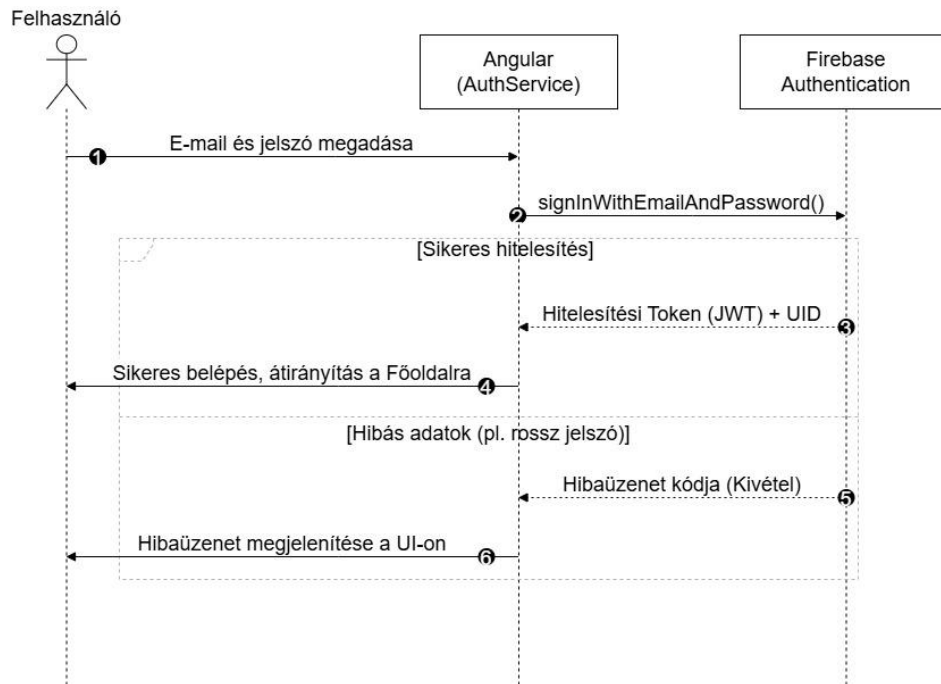


6.4.ábra. A fórum alapvető adatszerkezetének diagramja.

A fórum megnyitásakor az adott felhasználó adatait a rendszer a *ForumUser* adatszerkezetben tárolja. A fórum kollekcióját két alkollekcióra (karakterek és kalandok) lehet bontani. Ezekben a posztok dokumentumokként, a hozzájuk írt kommentek pedig egy újabb alkollekcióban, szintén dokumentumokként vannak eltárolva a **6.4. ábra** szerint.

7. Magas szintű folyamatok

7.1. Bejelentkezés az oldalra

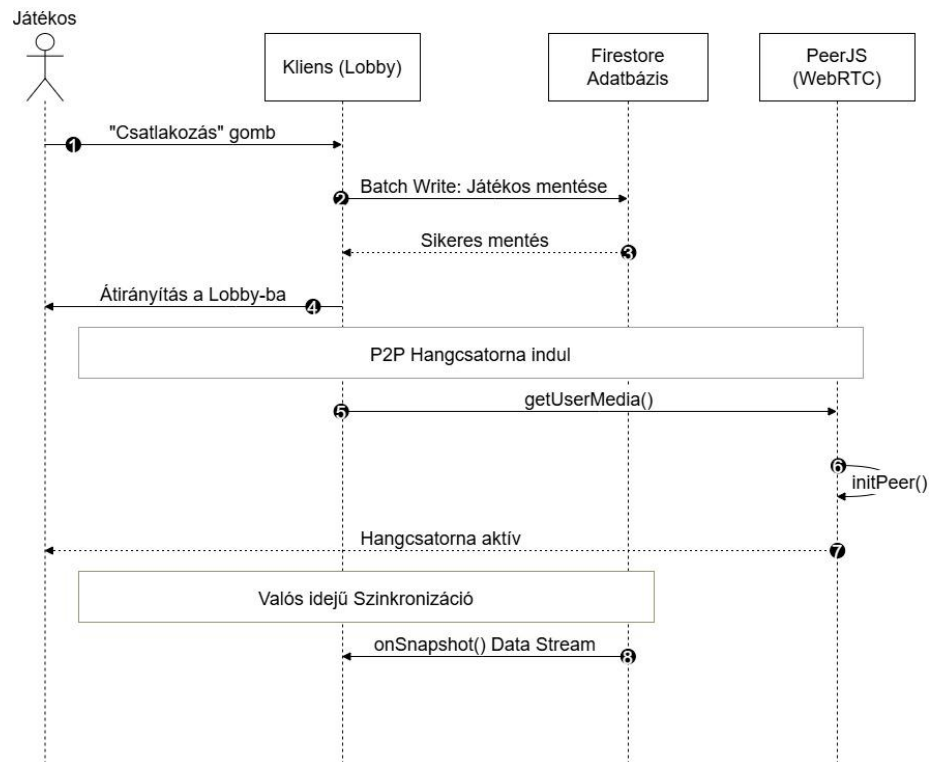


7.1. ábra. Felhasználó bejelentkezésének szekvenciadiagramja

A felhasználó azonosítását a Firebase Authentication végzi. Ez úgy lehetséges, hogy az alkalmazás az *AuthService* szolgáltatás segítségével átadja a Google szerverének a felhasználó által beírt email címet és jelszót. A jelszavak ellenőrzése a felhőben történik, ami után a rendszer sikeres hitelesítés esetén egy JSON Web Tokenet és egy egyedi azonosítót küld a kliensnek, ami ezek alapján tartja fent a munkamenetet és menedzseli a hozzáféréseket a különböző komponensekhez. A folyamatot a 7.1. ábra szemlélteti.

7.2 Játékhoz csatlakozás

A játékhoz csatlakozás két részből, egy adatbázis műveletből és egy hálózati kapcsolódásból épül fel. Elsőnek a rendszer a felhasználót egy játékos – **Melléklet X. táblázat.** – adatszerkezetben bele írja az adott játék játékos tömbjébe, ami után átirányítja a játék várótermébe (lobby).



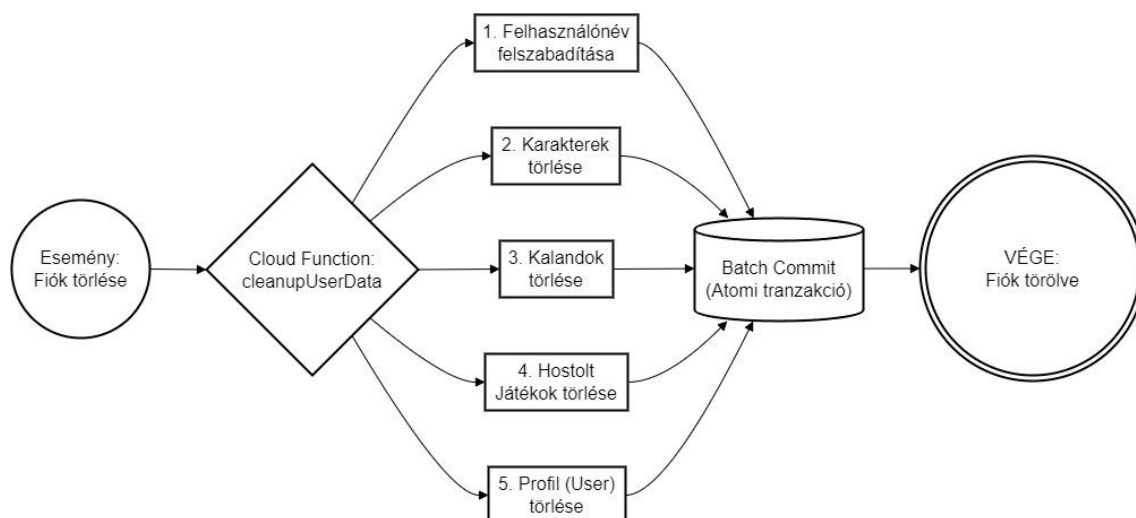
7.2. ábra. A játékhoz csatlakozás folyamatának szekvencia diagrammja

A csatlakozási folyamat második része a kliens böngészők kapcsolódása WebRTC API segítségével. Először a mikrofonengedélyeket kéri be a rendszer a böngésző MediaDevices API-ján keresztül, aztán a PeerJS Signalling szerverén[11] keresztül felépíti peer-to-peer kapcsolatot a többi játékkal. Ezzel párhuzamosan létrejön a **AngularFire docData()**[19] (*Firestore onSnapshot()*) metódusával egy adatfolyam a játék állapotának valós idejű szinkronizálásához minden csatlakozott kliensen.

7.3. Fiók törlése

A felhasználói fiók törlése egy esemény vezérelt serverless folyamat, ami a Google Cloud Functions szolgáltatást használja. A felhasználói fiók adatainak törlésekor fellépő esemény kiváltja a *cleanUpUserDataOnDelete()* – **Melléklet AT. kódrészlet** – felhőfüggvény lefutását egy admin környezetben, megkerülve a Security Rules-t.

Ez a függvény összegyűjti a felhasználóhoz tartozó referenciákat – felhasználó név, karakterek, kalandok, hosztolt játékok – és azokat egyetlen tranzakcióban (Batch Commit formájában) törli az adatbázisból, megőrizve közben az adatintegritást, hogy a folyamatban felmerülő hiba esetén az adatokat vissza lehessen állítani. A folyamatot a **7.3. ábra** szemlélteti.



7.3. ábra. A fióktörölés folyamatának eseménydiagramja

8. Fontosabb kódrészek

8.1. Karakterek / Kalandok létrehozása

A karakterek és kalandok létrehozásáért felelős logikák szinte megegyeznek egymással és csak kisebb részletekben különböznek. A létrehozási folyamatok a karakterek és kalandok adatszerkezetének létrehozásával kezdődnek, amit a szolgáltatásokban szereplő hozzáadó, az adatbázisba történő felvitelt végző függvény hívások követnek. –

8.1. kódrészlet:

```

1 const newCharacter: Character =
  createCharacter(this.mainForm, this.itemService)
2 await this.charService.addCharacter(newCharacter)
3 //-----//
4 const adventure: Omit<Adventure, 'id' | 'userId'> = {
5   name: this.adventureName.value,
6   events: this.events,
7 };
8 this.advService.addAdventure(adventure).then(...)
  
```

8.1. kódrészlet. Hozzáadás függvények hívása.

A szolgáltatások hozzáadás függvényeinek felépítése azonos, mindkettő aszinkron módon kerül végrehajtásra. Eltérés kizárólag a változónevekben és a Firestore-kollekciók azonosítójában mutatkozik. A függvények egy bemeneti értéket várnak, amit az előzőleg említett változók képviselnek. Az adatbázisba írás folyamata egy ellenőrzéssel kezdődik, ahol a felhasználó létezését ellenőrzi a rendszer az RxJS függvényeivel. – **8.2. kódrészlet:**

```
1 async addCharacter(character: Character):  
2 Promise<Character> {  
3   try {  
4     const user = await firstValueFrom(  
5       this.authService.currentUser.pipe(take(1)));  
6     if (!user) throw new Error('Felhasználó nem található!');  
7     const userDocRef = doc(this.firestore, 'Users', user.uid);  
8     const userSnap = await getDoc(userDocRef);  
9     if (!userSnap.exists())  
10    throw new Error('Felhasználó nem található!');  
11    const userData = userSnap.data() as User;
```

8.2. kódrészlet. Felhasználói adatok lekérése.

A sikeres ellenőrzést követően a felhasználó adatai a *userData* változóban tárolódnak el. A rendszer ezt használja fel a következő művelethez, ami az eltárolható karakterek/kalandok számának ellenőrzésére irányul. Ha lehetséges a mentés, akkor a rendszer lekér egy referenciát az adatbázis megfelelő kollekciójára és létrehozza a hozzáadandó elem dokumentumát. Ezután létrejön egy új adatszerkezet, ami a paraméterül adott adatok mellett a felhasználó azonosítóját (UID) és a frissen létrejött dokumentum azonosítóját is tartalmazza. – **8.3. kódrészlet:**

```
1 if (userData.characters.length === 10) {  
2   throw new Error('Maximum kaland szám elérve, nem  
3   készíthető több!');  
4 }  
5 const charactersColRef = collection(this.firestore,  
6 'Characters');  
7 const characterDocRef = doc(charactersColRef);  
8  
9 const newCharacter: Character = {  
10   ...character,  
11   id: characterDocRef.id,  
12   userId: user.uid,  
13 };
```

8.3. kódrészlet. Kollekciók lekérése és új karakterobjektum létrehozása.

A hozzáadási folyamatot az atomi művelet (Batch Commit) végrehajtása zárja, aminek során a végleges karakter/kaland adatszerkezete hozzárendelődik a dokumentum tartalmához, továbbá a felhasználó karakter/kaland azonosítóinak tárolójába az új dokumentum azonosítója is bejegyzésre kerül. – **8.4. kódrészlet:**

```
1 const batch = writeBatch(this.firestore);
2 batch.set(characterDocRef, newCharacter);
3 batch.update(userDocRef, {
4   characters: arrayUnion(characterDocRef.id),
5 });
6 await batch.commit();
7 return newCharacter;
8 } catch (error) {
9   console.error('Error adding character:', error);
10  throw error;
11 }
12 }
```

8.4. kódrészlet. Adatbázis művelet hívás és függvény lezárás.

Sikeres hozzáadás esetén a rendszer a felhasználót átirányítja az Angular Routing segítségével a profil oldalra, miközben attól függően, hogy karaktert vagy kalandot hozott létre a felhasználó, az adott konténert automatikusan megjeleníti. – **8.5. kódrészlet:**

```
1 localStorage.setItem('visibleContainerOnProfile',
2   'characters'
3 this.router.navigateByUrl('/profil');
```

8.5. kódrészlet. Navigáció a profiloldalra.

8.2. Csatlakozás a játékhoz

A játékhoz csatlakozás két fázisból áll. Elsőnek az adatbázis művelet fut le, ami beírja a játék adatszerkezetébe a felhasználót, majd a további játékosok felhívását végző WebRTC folyamatok következnek.

8.2.1. A csatlakozáshoz szükséges adatok feldolgozása

A játékhoz csatlakozás első fázisa egy aszinkron folyamat, ami a csatlakozáshoz szükséges adatok (kiválasztott karakter azonosítója és a játék azonosítója) meglétének ellenőrzésével kezdődik. – **8.6. kódrészlet:**

```

1  const characterId =
    this.joinGameForm.get('character')?.value;
2  if (!characterId) {
3      this.joinError = 'Válassz egy karaktert a
4      csatlakozáshoz!';
5      return;
6  }
7  if (id === null && this.joinGameForm.invalid) {
8      this.joinError = 'Töltsd ki a kötelező mezőket!';
9      return;
10 }
11 this.isLoading = true;
12 const gameId: string = id ? id :
    this.joinGameForm.get('joinID')?.value;
13 const selectedCharacter = this.myCharacters.find((c) =>
14 c.id === characterId);
15 if (!selectedCharacter) {
16     this.isLoading = false;
17     this.joinError = 'A kiválasztott karakter nem
18     található!';
19     return;
20 }

```

8.6. kódrészlet. Értékek beolvasása a UI űrlapjából.

Az ellenőrzés sikerességét követően meghívásra kerül a játékmenetért felelős szolgáltatásban (GameService) található *joinGame()* – **8.7. kódrészlet** – függvény. A csatlakozási folyamat a szolgáltatásban a létrehozási metódusokhoz hasonlóan a felhasználó ellenőrzésével kezdődik, amit a felhasználói- és a játék dokumentumok lekérése követ az adatbázisból. A lekérdezések után általános ellenőrzések mennek végbe a dokumentumok meglétéről. – **8.7. kódrészlet:**

```

1  async joinGame(gameId: string, selectedCharacter:
    Character): Promise<Game> {
2  try {
3  const user= await firstValueFrom(
4  this.authService.currentUser.pipe(take(1)));
5  if (!user) throw new Error('Felhasználó nem található!');
6  const userDocRef = doc(this.firestore, 'Users', user.uid);
7  const gameDocRef = doc(this.firestore, 'Games', gameId);
8  const [userSnap, gameSnap] = await Promise.all([
9      getDoc(userDocRef),
10     getDoc(gameDocRef),
11  ]);
12  if (!userSnap.exists())
13      throw new Error('Felhasználói profil nem található!');
14  if (!gameSnap.exists())
15      throw new Error('A játék nem található!');

```

8.7. kódrészlet. Felhasználói adatok ellenőrzése.

Miután az ellenőrzés lefutott, a dokumentumok adatait változókbba menti a rendszer és újabb ellenőrzések következnek. Először a játék tulajdonosának azonosítója és a felhasználó azonosítója kerül összehasonlításra, mivel egy játékot létrehozó (játékmester) nem vehet részt játékosként a saját játékában. – **8.8. kódrészlet:**

```

1  const userData = userSnap.data() as User;
2  const gameData = gameSnap.data() as Game;
3  if (gameData.ownerId === user.uid) {
4      throw new Error('Nem csatlakozhatsz a saját játékodhoz
        mint játékos!');
5  }

```

8.8. kódrészlet. Felhasználó és játékadatok elmentése.

A következő ellenőrzés a játékos már játékhoz csatlakozási státuszát vizsgálja meg. Ha a játékos már csatlakozva van a játékhoz, akkor a függvény visszaadja a játék adatait. Amennyiben egy másik játékhoz van már csatlakozva vagy a kiválasztott játékhoz a csatlakozás nem engedélyezett, akkor hibát dob. – **8.9. kódrészlet:**

```

1 if (userData.inGame === gameId) {return { ...gameData };}
2 else if (userData.inGame && userData.inGame !== gameId) {
3   throw new Error('Már csatlakozva vagy egy másik
   játékhoz!', { cause: GameErrorCauses.AlreadyInGame });
4 }
5 if (!gameData.isOpen) {
6   throw new Error('A csatlakozás nem engedélyezett (a
   játék zárva)!');
7 }
8 const currentPlayers = gameData.players || [];
9 if (currentPlayers.length >= gameData.maxPlayers) {
10   throw new Error('Nem lehet csatlakozni! Betelt a
   létszám.');
```

8.9. kódrészlet. Csatlakozási státuszellenőrzések.

Ha új csatlakozás szükséges, a függvényben létrejön a játékos adatszerkezete a *player* változóban. – 8.10. kódrészlet:

```

1 const player: Player = {
2   character: selectedCharacter,
3   status: PlayerStatus.NOTREADY,
4   initiative: null,
5   actionsLeft: { primary: true, secondary: true },
6   lastAction: {
7     performer: { id: '', name: '' },
8     primary: {} as GameAction,
9     secondary: {} as GameAction,
10  },
11  inCombat: false,
12  isVoting: false,
13  campActionPoints: 0,
14  name: userData.username ?? '',
15  id: user.uid,
16 };
```

8.10. kódrészlet. Játékos létrehozása.

A játékos létrehozását az adatok adatbázisba történő feltöltése követi, a korábban bemutatott karakter/kaland feltöltésével azonos módon. – 8.11. kódrészlet:

```

1  const batch = writeBatch(this.firestore);
2  batch.update(userDocRef, { inGame: gameId });
3  const updatedPlayers = [...currentPlayers, player];
4  batch.update(gameDocRef, { players: updatedPlayers });
5  await batch.commit();
6  return { ...gameData, players: updatedPlayers };
7  } catch (error: any) {
8      console.error('Join Error:', error);
9      throw error;
10  }
11 }

```

8.11. kódrészlet. Felhasználó és játékadatok elmentése.

Ha a csatlakozás sikeres, akkor a rendszer átirányítja a felhasználót a játék várótermébe az injektált router *navigate()* funkciójával. – **8.12. kódrészlet:**

```

1  if (g) {
2      this.gameService.PlayerRole = PlayerRole.PLAYER;
3      this.router.navigate(['/jatek', gameId, 'lobby']);
4  }

```

8.12. kódrészlet. Felhasználó átirányítása a játék oldalra.

8.2.2. Csatlakozás a hanghíváshoz

A csatlakozás második fázisa a hanghívás felépítése, ami már a váróteremben történik. A folyamat első lépése egy mikrofonhoz való hozzáférés engedélyeztetése a böngésző MediaDevices API-ján keresztül (ez felelős a JavaScript kód és a fizikai eszközök kapcsolatának felépítéséért). Ha a hozzáférés elutasításra kerül, akkor a folyamat megszakad és a hangcsatornához való csatlakozás kizárólag engedélyezés után folytatódhat – **8.13. kódrészlet:**

```

1  async getMicrophone(): Promise<MediaStream> {
2  try {
3    if (!navigator.mediaDevices ||
4      !navigator.mediaDevices.getUserMedia) {
5      throw new Error('A böngésző biztonsági okokból tiltja
6        a mikrofont (vagy nincs csatlakoztatva). Használj
7        HTTPS-t!');
8    }
9    const stream = await navigator.mediaDevices.
10     getUserMedia({ video: false, audio: true });
11    this.myMediaStream = stream;
12    return stream;
13 } catch (err) {
14     console.error('Hiba a mikrofon elérésekor:', err);
15     throw err;
16 }
17 }

```

8.13. kódrészlet. Mikrofon elérés engedélykérése.

Ha a felhasználó engedélyezte a mikrofont, a folyamat létrehozza a felhasználóhoz tartozó Peer objektumot. Ha már létezik egy aktív hívás az adott féllel, a függvény kilép, ha pedig érvénytelen a peer, akkor megsemmisíti, felszabadítva a memóriát – **8.14. kódrészlet:**

```

1  initPeer(userId: string) {
2    if (this.peer && !this.peer.disconnected &&
3      !this.peer.destroyed) return;
4    if (!this.myMediaStream) return;
5    if (this.calls.some(c => c.peer === userId)) return;
6    if (this.peer) this.peer.destroy();

```

8.14. kódrészlet. Felhasználói Peer létrehozása előtti ellenőrzések.

A függvény ezt követően csatlakozik a PeerJS Signalling szerverhez, hogy a felhasználó felvehesse a kapcsolatot a többi játékosal. Kapcsolat sikeres kiépítése esetén egy eseményfigyelő gondoskodik az adatfolyam kezeléséről. A bejövő hívások automatikus fogadása biztosítja, hogy a váróterembe újonnan belépő játékosok manuális beavatkozás nélkül kapcsolódhassanak a kommunikációba – **8.15. kódrészlet:**

```

7   this.peer = new Peer(userId);
8   this.peer.on('open', () => {
9     this.isConnected.next(true);
10  });
11
12  this.peer.on('error', (err) => {
13    console.error('PeerJS hiba:', err);
14  });
15
16  this.peer.on('call', (call) => {
17    call.answer(this.myMediaStream!);
18    this.handleCallStream(call);
19  });
20 }

```

8.15. kódrészlet. Peer létrehozása.

A peer létrehozása után a többi játékos felhívása következik. A folyamat két ellenőrzéssel indul, ahol a peer meglétét és a mikrofon bemenetet vizsgálja a rendszer (bármelyik nemléte esetén kilép) valamint azt, hogy az adott személlyel már folytat-e hívást a felhasználó. Amennyiben nincs hívás folyamatban, akkor felhívja a másik felet

– 8.16. kódrészlet:

```

1  connectToPeer(peerId: string) {
2    if (!this.peer || !this.myMediaStream) {
3      return;
4    }
5    if (this.calls.find(c => c && c.peer === peerId)) {
6      return;
7    }
8    const call = this.peer.call(peerId, this.myMediaStream);
9    if (call) {
10     this.handleCallStream(call);
11   } else {
12     console.warn(`Nem sikerült hívást kezdeményezni felé:
13     ${peerId} (A peer.call undefined-et adott vissza)`);
14   }
15 }

```

8.16. kódrészlet. Csatlakozás másik felhasználóhoz.

8.3. Támadás és reakció

Támadási akció végrehajtását kezdeményező játékos először egy sebzéskocka dobást hajt végre. Ekkor a célpont kapja meg az irányítást, ahol reagálhat a támadásra. Reakció után a támadó visszakapja a kezdeményezést, majd a folyamat kezdődik előlről.

8.3.1. Támadás

A támadás folyamata a játékos meglévő akcióinak és a kiválasztott célpont meglétének ellenőrzésével kezdődik. Hiányzó célpont esetén hibát dob a függvény és figyelmezteti a felhasználót, hogy válasszon egyet, ha pedig a böngésző helyi tárolójában nincsen elmentve a választott célpont, akkor elmenti azt. Ezt követően a célpont objektumot megkeresi a rendszer és egy változóban tárolja el. Célpont kiválasztása után a támadásra használt fegyver meglétét ellenőrzi a függvény – **8.17. kódrészlet:**

```
1  if ((!this.game?.camp.isCamping || this.isRaid()) &&
2      isPrimary && !this.player?.actionsLeft.primary) {
3      throw new Error('Nincs elsődleges akciód!');
4  } else if ((!this.game?.camp.isCamping || this.isRaid()) &&
5      !isPrimary && !this.player?.actionsLeft.secondary) {
6      throw new Error('Nincs másodlagos akciód!');
7  }if (!this.selectedTarget) {
8      throw new Error('Válassz ki egy célpontot.');
```

8.17. kódrészlet. Támadás előtti ellenőrzés és célpontmentés.

Ezután a játékos 3D-s felületen dobókockával határozza meg a sebzés mértékét a *rollForAttack()* – **Melléklet AF. kódrészlet** – függvénnyel. A dobás végeztével a játékostól levonásra kerül az adott akciópont. A játékos adatszerkezetében rögzítésre kerül és a játék jelenlegi elkövetett akcióihoz is hozzárendeli a *manageActions()* – **Melléklet AS. kódrészlet** – függvény a támadás akcióját – **8.18. kódrészlet:**

```
27 this.player.lastAction.primary = {
28   type: type,
29   target: this.selectedTarget?.id,
30   value: this.lastRoll,
31   reaction: { reacted: false },
32 };
33 this.game!.currentAction = this.player?.lastAction;
```

8.18. kódrészlet. Elsődleges akció elmentése.

A folyamat következő fázisa a megtámadott entitás reakciójának előkészítése. Ez a játék *playerOrder*, *prevPlayer* és *currentPlayer* változóinak módosításával valósul meg. A célpont – amennyiben még nem szerepel benne – bekerül a harci sorrendbe megtartva az integritást, majd a *prevPlayer* megkapja a támadó játékos azonosítóját, míg a *currentPlayer* a célpontét veszi át – **8.19. kódrészlet:**

```
1  if (!this.game?.playerOrder.find(p => p.id === target!.id))
2  {
3    this.game?.playerOrder.push({
4      id: target!.id,
5      initiative: Math.ceil(Math.random() * 20),
6      finished: false
7    })
8  }
9  this.player!.inCombat = true;
10 target!.inCombat = true;
11 this.game?.playerOrder.sort((a, b) => b.initiative -
12   a.initiative)
13 this.game!.prevPlayer = this.game?.currentPlayer!;
14 this.game!.currentPlayer = this.selectedTarget.id;
```

8.19. kódrészlet. Célpont játékosrendbe illesztése és irányítás átadás.

A támadási folyamatot az adatbázis művelet hívása zárja, ami egy *Partial<Game>*[6] adatszerkezetet – **Melléklet Z. táblázat** – vár. Ebbe a struktúrába azok az objektumok és állapotok kerülnek bele, amelyek a folyamat során ténylegesen megváltoztak – **8.20. kódrészlet:**

```

1 await this.gameService.updateGame(this.gameId, {
2   players: this.game?.players,
3   currentAction: this.game?.currentAction,
4   prevPlayer: this.game?.prevPlayer,
5   currentPlayer: this.game?.currentPlayer,
6   playerOrder: this.game?.playerOrder,
7   adventure: this.game?.adventure,
8   camp: this.game?.camp,
9 })

```

8.20. kódrészlet. Játékállapot mentése.

8.3.2. Reakció

A reakció egy reaktív mechanizmusra épül, ami a játék *currentAction* változójának értékváltozását monitorozza. Minden kliens figyeli ezt a változót, hogy van-e olyan támadás, aminek a célpontja (*target*) megegyezik az ő azonosítójával és még nem reagáltak rá. Ha ez a két feltétel teljesül, akkor van lehetősége a játékosnak négy reakció lehetőség közül választani – **8.21. kódrészlet:**

```

1 isReacting(): boolean {
2   let a = [this.game?.currentAction.primary,
3     this.game?.currentAction.secondary];
4   return
5     a.some(a => a?.target === this.player?.id && a?.type ===
6   ActionType.ATTACK && !a?.reaction!.reacted) && this.myTurn;
7 }
8 async reactToAttack(type: 'noReaction' | 'dodge' |
9   'attackBack' | 'parry')

```

8.21. kódrészlet. Reakció lehetőség ellenőrzése.

A reakció folyamata egy ellenőrzéssel kezdődik, ami a karakter stressz pontjait figyeli, ami, ha 1 értéket vesz fel, akkor egy figyelmeztetéssel kerül szembe a játékos, hogy a karaktere az örület szélén áll és folytatás esetén megőrül. Ha a játékos folytatja, akkor a *manageDeathAndInsane()* – **Melléklet AM. kódrészlet** – függvény hozzáadja az „Őrült” jelzőt a karakterhez, kiveszi a játékos sorrendből és szinkronizálja a játék dokumentumát az adatbázissal – **8.22. kódrészlet:**

```

1  if (this.player?.character?.stats.main.sp === 1 && type !==
2  'noReaction') {
3    if (!confirm('A karaktered a megőrülés szélén áll.
4    Biztosan folytatod?')) {
5      return;
6    }
7    await this.manageDeathAndInsane('insane', this.player);
8    await this.gameService.updateGame(this.gameId, {
9      players: this.game?.players,
10     camp: this.game?.camp,
11     adventure: this.game?.adventure,
12   });
13   this.openSnackBar('A karaktered megőrült.');
```

8.22. kódrészlet. Karakter megőrülésének ellenőrzése és kezelése.

Ezután eltárolásra kerül a támadó, létrehozásra kerül egy változó a reakció sikerének eltárolására és kiszámítódik a sebzés, amit a célpont elszenvedhet (a bejövő sebzés – a karakter páncéljának védelmi értéke) – **8.23. kódrészlet:**

```

1  const attacker = this.game?.players.find(p => p.id ===
2  attackerId) ?? this.game?.camp?.raid.find(r => r.id ===
3  attackerId) ?? this.currentEvent?.NPCs.find(n => n.id
4  === attackerId) ?? null;
5  this.selectedTarget = attacker ?? null;
6  if (!attacker) {
7    throw new Error('Nem található a támadó.');
```

8.23. kódrészlet. Támadó objektum megkeresése és sebzésszámítás.

A játékos választásától függően négy függvény futhat le – **8.24. kódrészlet.** Ha nincs reakció: *applyDamage()* – **Melléklet AN. kódrészlet,** kikerülés esetén *dodgeAttack()* – **Melléklet AP. kódrészlet,** visszatámadás esetén *attackBack()* – **Melléklet AQ. kódrészlet,** hárítás esetén *parryAttack()* – **Melléklet AR. kódrészlet:**

```

1 switch (type) {
2   case 'noReaction':
3     this.openSnackBar(`Nincs reakció. ${dmgToDefender > 0
4       ? `Elszenvedsz -${dmgToDefender} HP sebzést.` : 'Nem
5       szenvedsz sérülést.'`);
6     await this.applyDamage(damage, this.player!);
7     break;
8   }

```

8.24. kódrészlet. Reakció kezelése switch-el.

Ha sikeres visszatámadás/hárítás volt a reakció, akkor a *reactSuccess* egy objektum lesz, ezért egy ellenőrzésre van szükség a reakció objektumának pontos felépítéséhez. Ezután attól függően, hogy elsődleges vagy másodlagos akcióra reagáltak, létrejön a reakció objektuma, ezt követően pedig az adatbázis művelet – **8.25. kódrészlet:**

```

1 const isComplexResult = reactSuccess !== null && typeof
  reactSuccess === 'object';
2 if (this.attackActionIsPrimary) {
3   const primaryReaction = {
4     reacted: true,
5     reactionType: reactionType,
6     success: isComplexResult ? (reactSuccess as
7       any).success : (reactSuccess ?? false),
8     counterDamage: isComplexResult ? (reactSuccess as
9       any).dmg : counterDmg,
10  }
11  this.game!.currentAction.primary.reaction =
12    {...primaryReaction };
13  attacker!.lastAction.primary!.reaction =
14    {...primaryReaction };
15 }
16 await this.gameService.updateGame(this.gameId, {
17   players: this.game?.players,
18   camp: this.game?.camp,
19   currentAction: this.game?.currentAction,
20   playerOrder: this.game?.playerOrder,
21   currentPlayer: this.game?.prevPlayer ?? '',
22   prevPlayer: '',
23   adventure: this.game?.adventure,
24 })

```

8.25. kódrészlet. Reakció objektum meghatározása és adatbázis frissítés.

9. Tapasztalatok, továbbfejlesztési lehetőségek

9.1. Továbbfejlesztési lehetőségek

Mivel a szerepjáték rendszere még fejlesztés alatt áll, így a folyamatos igazítások a meglévő funkciókban elengedhetetlenek. A dolgozat írásának ebben a pillanatában is vannak már változások például a harc rendszer működésében, de idő hiányában nem kerültek implementálásra.

Elengedhetetlen a rendszer oldalát bemutató könyv formátum újra dolgozása egy olyan másik formátumra, amit könnyebb szerkeszteni és karban tartani. Emellett szükségesnek gondolom, hogy a világtérképet is átgondoljam és részletesebben újra alkossam.

Fontosnak tartom a játéktér kibővítését úgy, hogy a kaland vizuálisan is értelmezhető legyen, akár csak a D&D Beyond, Roll20 és Foundry VTT rendszerében. A játékosok karakterei a térképen helyezkedjenek el a többi NPC-vel és más olyan lényvel, amivel a játékosok találkozhatnak. Ebből adódóan további a kalandhoz hozzáadható lényt, épületet, tárgyat is jó ötlet lenne lefejleszteni.

További fejlesztést igényelhet még a karakterek és kalandok megtekintési lehetősége, azon felül, ami a profil oldalon lehetséges. Ez egy teljesen új oldalon történne, ahol szükség esetén még szerkeszteni is lehetne utólagosan a karaktereket és a kalandokat.

A fórum oldalára érdemes lenne több alfórumot bevezetni, elkülöníteni a szerepeket, illetve egyéb alfórumokat is bevezetni (pl. hibákkal kapcsolatos észrevételek fóruma).

9.2. Tapasztalatok

A fejlesztési folyamat összegésében gördülékenyen folyt. Számos új dologgal találkoztam. Eddig nem volt szükséges ennyire komolyan megterveznem egy programot és annak ellenére, hogy a téma már ismerős volt, fejlesztés közben kifejezetten hasznos volt, hogy az alap koncepció a képernyő tervek által már létezik és csak apró javításokat kellett végezni.

A fejlesztési folyamat felénél felfedeztem a Firebase emulátorokat, így a teszteléskor nem az ingyenes keretet kellett elhasználnom Firebase-ben és nem az éles

adatbázist szemeteltem tele a tesztelés során. Ez volt az első találkozásom a Cloud Functions szolgáltatással, aminek a használata a kezdeti fázisokban még nem volt lehetséges, de a kliens „megbízhatatlanságának elve” elvezetett a bevezetéséhez a fórum fejlesztésekor, amikor a felhasználó törlésének implementálását végeztem.

A Github felületének sokkal nagyobb kiaknázását érdemes lenne fejlesztenem, mert egy szakmai környezetben nem teljesen helytállóak azok az *Issue*-k, amiket írtam. Valamint az API kulcsok biztonságát is hasznos lenne tanulmányoznom annak ellenére, hogy a projekt egy publikus API kulcsot használ.

Irodalomjegyzék

1. Cloud Firestore – <https://firebase.google.com/docs/firestore> – 2026.03.30.
2. Firebase Authentication – <https://firebase.google.com/docs/auth> – 2026.03.30.
3. Firebase Security Rules – <https://firebase.google.com/docs/rules> – 2026.03.30.
4. Firebase Hosting – <https://firebase.google.com/docs/hosting> – 2026.03.30.
5. What is Angular? – <https://angular.dev/overview> – 2026.03.30.
6. TypeScript Documentation – <https://www.typescriptlang.org/docs/> – 2026.03.30.
7. RxJS Operators – <https://rxjs.dev/guide/overview> – 2026.03.30.
8. Leaflet API reference – <https://leafletjs.com/reference.html> – 2026.03.30.
9. Getting started with peer connections – <https://webrtc.org/getting-started/peer-connections> – 2026.03.30.
10. WebRTC Signaling and video calling – https://developer.mozilla.org/en-US/docs/Web/API/WebRTC_API/Signaling_and_video_calling – 2026.03.30.
11. PeerJS Getting started – <https://peerjs.com/client/getting-started> – 2026.03.30.
12. Web Audio API – https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API – 2026.03.30.
13. MediaDevices – <https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices> – 2026.03.30.
14. Angular Routing – <https://angular.dev/guide/routing> – 2026.03.30.
15. Dependency Injection in Angular – <https://angular.dev/guide/di> – 2026.03.30.
16. Accepting data with input properties – <https://angular.dev/guide/components/inputs> – 2026.03.30.
17. Custom events with outputs – <https://angular.dev/guide/components/outputs> – 2026.03.30.
18. Cloud Functions for Firebase – <https://firebase.google.com/docs/functions> – 2026.03.30.
19. AngularFire – <https://github.com/angular/angularfire?tab=readme-ov-file#readme> – 2026.03.30.
20. Ébredő világok szerepjáték dokumentáció – <https://drive.google.com/file/d/1D8P4JJusi2vaBkG-H0HKfv2MfsZOvAPi/view?usp=sharing> – 2026.03.30.

Nyilatkozat

Alulírott Gyurcsó-Licsman Péter programtervező informatikus BSc szakos hallgató, kijelentem, hogy a dolgozatomat a Szegedi Tudományegyetem, Informatikai Intézet Szoftverfejlesztési Tanszékén készítettem, programtervező informatikus BSc diploma megszerzése érdekében.

Kijelentem, hogy a dolgozatot más szakon korábban nem védtem meg, saját munkám eredménye, és csak a hivatkozott forrásokat (szakirodalom, eszközök, stb.) használtam fel.

Tudomásul veszem, hogy szakdolgozatomat / diplomamunkámat a Szegedi Tudományegyetem Diplomamunka Repozitóriumban tárolja.

Szeged, 2026. május 10.

Gyurcsó-Licsman Péter

Aláírás

Nyilatkozat az mesterséges intelligencia használatáról

Alulírott Gyurcsó-Licsman Péter kijelentem, hogy az „Ébredő Világok, az online szerepjáték platform” című szakdolgozatom részét képező szoftveralkalmazás fejlesztése során generatív mesterséges intelligencia (MI) alapú eszközöket és asszisztenseket vettem igénybe.

Ezúton nyilatkozom az MI felhasználatának mértékéről:

Specifikus modulok esetében mesterséges intelligencia segítségével generáltam, egészítettem ki a kódalapot. Az MI használata az alábbi funkcionális egységekre korlátozódott:

- **3D dobókocka-szimuláció:** A Three.js grafikus és a Cannon-es fizikai könyvtárak integrációja, a kockák térbeli mozgásának, ütközéseinek, valamint a dobott értékek kiszámításának logikai megtervezése.
- **WebRTC audiokommunikáció:** A PeerJS könyvtárra épülő hangkapcsolat felépítésének megtervezése.
- **Firebase Cloud Functions:** A háttérben futó, eseményvezérelt adatbázis-takarító algoritmusok az ütemezett takarítási feladatok szerveroldali logikájának kiegészítése.
- **Firestore Security Rules:** A adatbázis hozzáférés-kezelési deklarációinak, a jogosultságok érvényesítési szabályrendszerének kiegészítése.

Szeged, 2026. május 10.

Gyurcsó-Licsman Péter

Aláírás

Mellékletek



10.1. ábra. Github repository QR kód

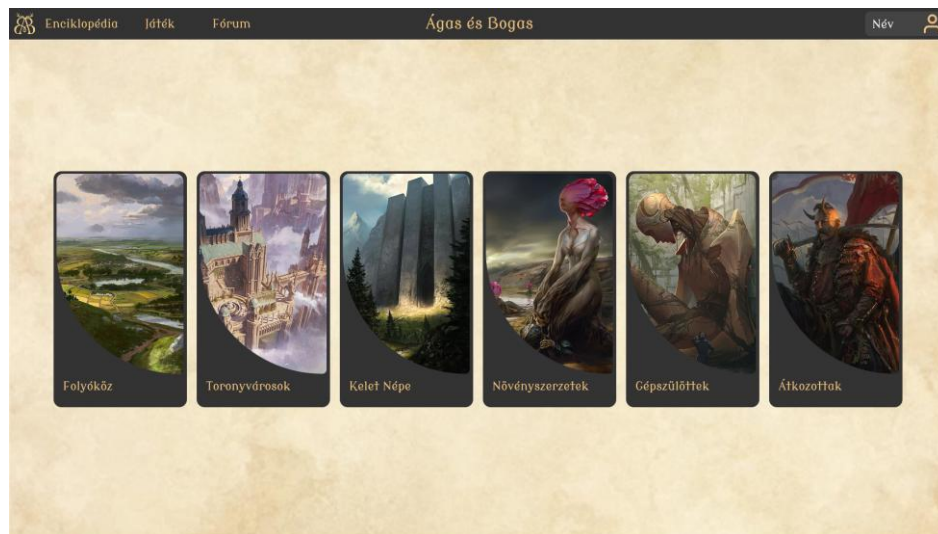
<https://github.com/TheChillMan25/EbredoVilagok>

A. Világ oldal



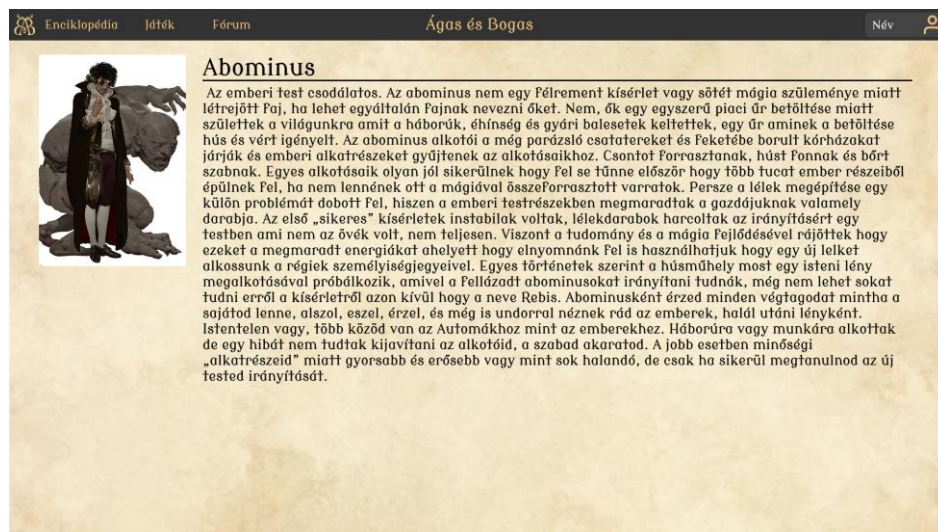
A. ábra. A világ képernyőterve.

B. Fajok és Népek oldal



B. ábra. A fajok főoldalának képernyőterve.

C. Faj oldal



C. ábra. A fajok listázásának képernyőterve.

D. Térkép oldal



D. ábra. A térkép oldal képernyőterve.

E. Rendszer oldal



E. ábra. A rendszer oldal képernyőterve.

F. Aktív státuszhatások

ActiveStatus		
<i>type</i>	StatusType	Státuszhatás típusa
<i>duration</i>	number	Hatás hossza körökben
<i>value?</i>	number	Hatás módosító értéke

F. táblázat. Státuszhatás adatszerkezet.

G. Státuszhatás típusok

StatusType	
BLEED	BLEED
POISON	POISON
BURN	BURN
LOST_LIMB	LOST_LIMB
PROSTHETIC	PROSTHETIC
FIRE_RES	FIRE_RES
POISON_RES	POISON_RES
ADVANTAGE	ADVANTAGE
DISADVANTAGE	DISADVANTAGE
ANIMAL_TOUNGE	ANIMAL_TOUNGE
FREE_MAGIC	FREE_MAGIC
SLEEP	SLEEP
STRESS_RES	STRESS_RES
FULL_BELLY	FULL_BELLY
DEAD	DEAD
INSANE	INSANE

G. táblázat. Státuszhatás típusai (enum).

H. Tárgy

Item		
<i>id</i>	number	Azonosító
<i>name</i>	string	Tárgy neve
<i>desc</i>	string	Tárgy leírása
<i>size</i>	SMALL / NORMAL / LARGE	Tárgy mérete
<i>type</i>	ItemType	Tárgy típusa
<i>category</i>	ItemCategory	Tárgy kategóriája
<i>uses?</i>	number	Használati érték
<i>effects?</i>	array<ItemEffect>	Adható státuszok
<i>price?</i>	number	Ár

H. táblázat. Tárgy adatszerkezet.

I. Tároló

Inventory		
<i>canStore</i>	array<ItemType>	Tárolható típusok
<i>storage</i>	array<{id: string, amount: number}>	Tárolt tárgyak
<i>space</i>	number	Max. tárolható tárgyak

I. táblázat. Tároló adatszerkezet.

J. Különleges tárgyak

SpecialItem		
<i>effectDesc</i>	string	Különleges hatás leírása
<i>isPartyWide</i>	boolean	Egész csapatra ható
<i>combat</i>	boolean	Harc közben használható

J. táblázat. Különleges tárgy adatszerkezet.

K. Élelem

Food		
<i>heal</i>	number	Gyógyítás az elfogyasztáskor

K. táblázat. Élelem adatszerkezet.

L. Fegyver

Weapon		
<i>diceCount</i>	number	Sebzés kockák száma
<i>damage</i>	string	Sebzés kocka típusa
<i>weaponType</i>	'melee' / 'ranged'	Fegyver típusa
<i>price</i>	number	Ár
<i>handed</i>	0 / 1 / 2	Fegyver tartásához szükséges kezek száma
<i>reload</i>	boolean	Újratöltés szükséges

L. táblázat. Fegyver adatszerkezet.

M. Páncél

Armour		
<i>defValue</i>	number	Védelmi érték
<i>dexMod</i>	number	Ügyesség módosító
<i>price</i>	number	Ár

M. táblázat. Páncél adatszerkezet.

O. Szivar

Cigar		
color	string	Szivar színe
spice	string	Szivar fűszerezése
effectDesc	string	Különleges hatás leírása

O. táblázat. Szivar adatszerkezet.

P. Tárgy típusok

ItemType	
COMMON	COMMON
FOOD	FOOD
MEDICAL	MEDICAL
CIGAR	CIGAR
SPECIAL	SPECIAL
WEAPON	WEAPON
ARMOUR	ARMOUR
AMMO	AMMO
SPICE	SPICE
BAG	BAG

P. táblázat. Tárgy típusok (enum).

Q. Tárgy kategóriák

ItemCategory	
EQUIPMENT	EQUIPMENT
CONSUMABLE	CONSUMABLE
GENERAL	GENERAL
INVENTORY	INVENTORY

Q. táblázat. Tárgy kategóriák (enum).

R. Tárgy hatás

ItemEffect		
<i>type</i>	EffectType	Hatás típusa
<i>value?</i>	number	Módosító érték
<i>stat?</i>	string	Módosítandó tulajdonság
<i>status?</i>	StatusType	Hozzáadandó / elvetendő státuszhatás
<i>duration?</i>	number	Hatás időtartama
<i>target</i>	'self' / 'target' / 'party'	Hatás célpontja

R. táblázat. Tárgy hatás adatszerkezet.

S. Hatás típusok

EffectType	
HEAL_HP	HEAL_HP
HEAL_SP	HEAL_SP
HEAL_SMALL_WOUND	HEAL_SMALL_WOUND
HEAL_LARGE_WOUND	HEAL_LARGE_WOUND
BUFF_STAT	BUFF_STAT
REMOVE_STATUS	REMOVE_STATUS
ADD_STATUS	ADD_STATUS

S. táblázat. Hatás típusok (enum).

T. A kaland eseményei

AdventureEvent		
<i>id</i>	number	Azonosító
<i>name</i>	string	Esemény neve
<i>desc</i>	string	Esemény leírása
<i>story</i>	string	Esemény története
<i>location</i>	string	Helyszín
<i>completed</i>	boolean	Esemény teljesítve van
<i>NPCs</i>	array<NPC>	Az NPC-k listája

T. táblázat. Kaland esemény adatszerkezet.

U. Publikus karakter

A *Character* adatszerkezetből az *id*, *currentAdventure* és *userId* mező elhagyásával azonos az eredetivel.

V. Publikus kaland

Az *Adventure* adatszerkezetből az *id* és *userId* mezőt elhagyva megegyezik az eredetivel.

W. Játék résztvevők / szereplők

GameParticipant		
id	string	Azonosító
name	string	Név
character	Character	A résztvevő karaktere
initiative	number null	A szereplő sorrendje
inCombat	boolean	A szereplő harcban
actionsLeft.primary	boolean	Hátralévő elsődleges akció
actionsLeft.secondary	boolean	Hátralévő másodlagos akció
lastAction.primary	GameAction	Elsődlegesen végrehajtott akció
lastAction.secondary	GameAction	Másodlagosan végrehajtott akció
performer.id	string	Végrehajtó azonosítója
performer.name	string	Végrehajtó neve

W. táblázat. Játékrésztvevő adatszerkezet.

X. Játékos

Player		
status	READY / NOTREADY	Készenléti státusz
isVoting	boolean	A játékos szavaz
campActionPoints	number	Táborozási akciópontok

X. táblázat. Játékos adatszerkezet.

Y. Nem játékos karakter (Non-Player character – NPC)

NPC		
attitude	'neutral' / 'hostile'	Viselkedésmód / hozzáállás
isTrader	boolean	Az NPC kereskedő
trades	array<Record<string,	Árusított tárgyak

	{pieces: number, item: Food SpecialItem Cigar}>>	
isVisible	boolean	Esemény kezdetekor látható

X. táblázat. NPC adatszerkezet.

Z. A játék adatszerkezete

Game		
<i>id</i>	string	Azonosító
<i>ownerName</i>	string	Játékot létrehozó neve
<i>ownerId</i>	string	Játékot létrehozó azonosítója
<i>maxPlayers</i>	number	Max. részt vehető játékosok száma
<i>playerOrder</i>	array<{ id: string, initiative: number, finished: boolean}>	Játékos sorrend – {játékos azonosító, sorrendi érték, befejezte körét}
<i>players</i>	array<Player>	A játékhoz csatlakozott játékosok
<i>prevPlayer</i>	string	Előző játékos azonosítója
<i>currentPlayer</i>	string	Éppen körön lévő játékos
<i>currentAction.primary</i>	GameAction	Végrehajtott elsődleges akció
<i>currentAction.secondary</i>	GameAction	Végrehajtott elsődleges akció
<i>currentAction.performer.id</i>	string	Akciót végrehajtó azonosítója
<i>currentAction.performer.name</i>	string	Akciót végrehajtó neve
<i>currentEvent</i>	number	Soron lévő esemény azonosítója
<i>adventure</i>	Adventure	A játékhoz tartozó kaland
<i>isOpen</i>	boolean	Csatlakozás lehetséges
<i>isPublic</i>	boolean	Publikus a játék
<i>started</i>	boolean	Elkezdődött a játék
<i>vote.theme</i>	string	Szavazás témája
<i>vote.starter</i>	string	Szavazás kezdeményezője
<i>vote.votes</i>	array<{	Szavazatok – {játékos neve, játékos

	player: string, vote: boolean }>	szavazata}
<i>camp.isCamping</i>	boolean	A játékosok táboroznak
<i>camp.raid</i>	array<NPC>	Táborozás rajtaütői
<i>camp.campActions.fire</i>	number	Tűzgyújtás kötelező akció értéke
<i>camp.campActions.tents</i>	number	Sátor állítás kötelező akció értéke
<i>camp.campActions.traps</i>	number	Csapda állítás kötelező akció értéke
<i>camp.campActions.guard</i>	number	Őrködés kötelező akció értéke

Z. táblázat. A „Játék” teljes adatszerkezete.

AA. Játékakció

GameAction		
<i>type</i>	ActionType MandatoryCampAction StandardCampAction	Akció típusa
<i>target?</i>	string	Akció célpontja
<i>item?</i>	number	Használt tárgy azonosítója
<i>value?</i>	number	Módosító érték
<i>reaction.react</i>	boolean	Reagáltak az akcióra
<i>reactionType</i>	Reaction	Reakció típusa
<i>success</i>	boolean null	Sikeres-e a reakció
<i>counterDamage</i>	number	Ellentámadási érték

AA. táblázat. Játékbeli akció adatszerkezet.

AB. Akció típusok

ActionType	
USEITEM	USEITEM
CAMP	CAMP
TALK	TALK
TRADE	TRADE

ATTACK	ATTACK
LOOT	LOOT
NONE	NONE

AB. táblázat. Akció típusok (enum).

AC. Kötelező tábori akciók

MandatoryCampActions	
START_FIRES	START_FIRES
SET_UP_TENTS	SET_UP_TENTS
SET_UP_TRAPS	SET_UP_TRAPS
GUARD	GUARD

AC. táblázat. Kötelező tábori akció típusok.

AD. Általános tábori akciók

StandardCampActions	
TREAT_WOUNDS	TREAT_WOUNDS
CALM_OTHERS	CALM_OTHERS
GATHER_PLANTS	GATHER_PLANTS
HUNT	HUNT

AD. táblázat. Általános tábori akció típusok.

AE. Játéktér hibák

GameErrorCauses	
NoPrimaryAction	0
NoSecondaryAction	1
HPAlreadyFull	2
SPAlreadyFull	3
NoSmallWounds	4
NoLargeWounds	5
NoStatToBuff	6
NoStatusToAdd	7
NoStatusToRemove	8
NPCInCombat	9
NoPermission	10
NPCNotFound	11
GameUpdateError	12
AlreadyInGame	13

AE. táblázat. Játéktéri hibatípusok (enum).

AF. rollForAttack() – A támadás kockadobása

```
1  async rollForAttack(diceCount: number | undefined, damage:
    string | undefined) {
2      if (!this.myTurn) return;
3      if (!diceCount || !damage) return;
4      let dices = [];
5      for (let i = 0; i < diceCount; i++) {
6          dices.push(damage);
7      }
8      await this.openDiceRoller(dices);
9      this.openSnackBar(`Támadás
    "${this.selectedTarget?.name}" ellen "${this.lastRoll}"
    sebzéssel.`)
10 }
```

AF. kódrészlet. rollForAttack() függvény.

AG. openDiceRoller() – Kocka dobás előkészület

```
1 openDiceRoller(dice: string[] = ['d20']): Promise<number> {
2   this.diceToRoll = dice;
3   this.showDiceRoller = true;
4   return new Promise<number>((resolve) => {
5     this.diceRollResolver = resolve;
6   })
7 }
```

AG. kódrészlet. openDiceRoller() függvény.

AH. throwDice() – Kocka dobás

```
1 throwDice() {
2   if (this.isRolling) return;
3   this.isRolling = true;
4   this.resultText = '';
5   this.rollStartTime = Date.now();
6   const dist = this.camera.position.y;
7   const fovRad = (this.camera.fov * Math.PI) / 180;
8   const visibleHeight = 2 * Math.tan(fovRad / 2) * dist;
9   const aspect = window.innerWidth / window.innerHeight;
10  const visibleWidth = visibleHeight * aspect;
11  const minDimension = Math.min(visibleWidth,
12                                visibleHeight);
13  const throwRadius = minDimension * 0.4;
14  const throwHeight = 3;
15  this.diceObjects.forEach((obj, i) => {
16    obj.stopped = false;
17    obj.stableCount = 0;
18    obj.body.allowSleep = true;
19    obj.body.sleepSpeedLimit = 0.08;
20    obj.body.sleepTimeLimit = 0.7;
21    obj.body.wakeUp();
22    const angle = Math.random() * Math.PI * 2;
23    const startX = Math.cos(angle) * throwRadius;
24    const startZ = Math.sin(angle) * throwRadius;
25    obj.body.position.set(startX, throwHeight + i * 1.5,
26                          startZ);
27    const targetX = Math.random() * 4 - 2;
28    const targetZ = Math.random() * 4 - 2;
29    const velocityX = targetX - startX;
30    const velocityZ = targetZ - startZ;
31    const distance = Math.sqrt(velocityX * velocityX +
32                               velocityZ * velocityZ);
33    const speedMultiplier = distance * 2;
```

```

34     const velX = (velocityX / distance) *
35                                     speedMultiplier;
36     const velZ = (velocityZ / distance) *
37                                     speedMultiplier;
38     obj.body.velocity.set(velX, 10, velZ);
39     obj.body.angularVelocity.set(
40         Math.random() * 10 - 5,
41         Math.random() * 10 - 5,
42         Math.random() * 10 - 5,
43     );
44     obj.body.quaternion.setFromEuler(
45         Math.random() * Math.PI,
46         Math.random() * Math.PI,
47         Math.random() * Math.PI,
48     );
49     obj.mesh.position.copy(obj.body.position as any);
50     obj.mesh.quaternion.copy(obj.body.quaternion as any);
51 });
52 this.checkRollingStatus();
53 }

```

AH. kódrészlet. throwDice() függvény.

AI. checkRollingStatus() – Dobási folyamat figyelése

```

1 private checkRollingStatus() {
2     const checkInterval = setInterval(() => {
3         if (Date.now() - this.rollStartTime < 1500) {
4             return;
5         }
6         let allStopped = true;
7         this.diceObjects.forEach((obj) => {
8             if (obj.stopped) return;
9             const isSleeping = obj.body.sleepState ===
10                                CANNON.Body.SLEEPING;
11             const speed = obj.body.velocity.length();
12             const angularSpeed =
13                 obj.body.angularVelocity.length();
14             const isSlow = speed < 0.05 && angularSpeed < 0.05;
15
16             if (isSleeping || isSlow) obj.stableCount++;
17             else obj.stableCount = 0;
18             if (obj.stableCount > 10) obj.stopped = true;
19             else allStopped = false;
20         });
21
22         if (allStopped) {
23             if (this.allStoppedSince === null) {

```

```

24         this.allStoppedSince = Date.now();
25         return;
26     }
27
28     if (Date.now() - this.allStoppedSince >=
29         this.finalStableWindowMs) {
30         clearInterval(checkInterval);
31         requestAnimationFrame(() => this.finishRoll());
32     }
33 } else {
34     this.allStoppedSince = null;
35 }
36 }, 50);
37 setTimeout(() => {
38     clearInterval(checkInterval);
39     if (this.isRolling) this.finishRoll();
40 }, 5000);
41 }

```

AG. kódrészlet. checkRollingStatus() függvény.

AJ. rollFinished() – Dobás befejezésének jelzése

```

1 private finishRoll() {
2     if (!this.isRolling) return;
3     this.isRolling = false;
4     const results = this.diceObjects.map((obj) => {
5         return this.calculateRealResult(obj);
6     });
7
8     let finalRoll = 0;
9     let details = '';
10
11     if (this.diceCountModifier === 'adv') {
12         const maxVal = Math.max(...results);
13         finalRoll = maxVal + this.rollModifier;
14         details = `(Adv: [${results.join(', ')}])`;
15     } else if (this.diceCountModifier === 'disadv') {
16         const minVal = Math.min(...results);
17         finalRoll = minVal + this.rollModifier;
18         details = `(Disadv: [${results.join(', ')}])`;
19     } else {
20         const sum = results.reduce((a, b) => a + b, 0);
21         finalRoll = sum + this.rollModifier;
22         details = `[${results.join(' + ')}]`;
23     }
24     this.resultText = `${finalRoll} ${details}
25     ${this.rollModifier ? '+ ' + this.rollModifier : ''}`;

```

```
26 this.rollFinished.emit(finalRoll);
27 }
```

AJ. kódrészlet. rollFinished() függvény.

AK. calculateRealResult() – Dobott értéklapindex számítása

```
1 private calculateRealResult(obj: { body: CANNON.Body;
2                                     type: string; }): number {
3     const body = obj.body;
4     const floorVector = new CANNON.Vec3(0, -1, 0);
5     let faceNormals: CANNON.Vec3[] = [];
6     if (obj.type === 'd6') {
7         faceNormals = [
8             new CANNON.Vec3(1, 0, 0),
9             new CANNON.Vec3(-1, 0, 0),
10            new CANNON.Vec3(0, 1, 0),
11            new CANNON.Vec3(0, -1, 0),
12            new CANNON.Vec3(0, 0, 1),
13            new CANNON.Vec3(0, 0, -1),
14        ];
15     } else {
16         const shape = body.shapes[0] as CANNON.ConvexPolyhedron;
17         faceNormals = shape.faceNormals;
18     }
19     let maxDot = -Infinity;
20     let bestFaceIndex = -1;
21     let bestWorldNormal = new CANNON.Vec3();
22     for (let i = 0; i < faceNormals.length; i++) {
23         const worldNormal =
24             body.quaternion.vmult(faceNormals[i]);
25         const dot = worldNormal.dot(floorVector);
26         if (dot > maxDot) {
27             maxDot = dot;
28             bestFaceIndex = i;
29             bestWorldNormal = worldNormal;
30         }
31     }
32     return this.getFaceMap(obj.type, bestFaceIndex) > 1 ?
33         this.getFaceMap(obj.type, bestFaceIndex) : 1;
34 }
```

AK. kódrészlet. calculateRealResult() függvény.

AL. getFaceMap() – Dobás értékének meghatározása

```
35 private getFaceMap(type: string, faceIndex: number): number
36 {
37     const maps: { [key: string]: number[] } = {
38         d4: [2, 3, 1, 4],
39         d6: [1, 6, 2, 5, 3, 4],
40         d8: [5, 6, 1, 7, 3, 4, 2, 8],
41         d10: [4, 9, 4, 3, 10, 6, 4, 7, 2, 5, 2, 5, 8, 7, 6, 1,
42             4, 10, 6, 9],
43         d12: [12, 1, 1, 2, 2, 8, 7, 3, 10, 10, 10, 10, 5, 5, 5,
44             4, 4, 6, 7, 7, 2, 8, 6, 8, 1, 9, 1, 3, 3, 6, 11,
45             11, 11, 9, 12, 9],
46         d20: [6, 13, 2, 10, 3, 9, 15, 12, 14, 11, 4, 19, 16,
47             18, 8, 17, 7, 1, 5, 20],
48     };
49     const map = maps[type] || [];
50     return map[faceIndex] !== undefined ? map[faceIndex] :
51         faceIndex + 1;
52 }
```

AL. kódrészlet. getFaceMap() függvény.

AM. manageDeathAndInsane() – Halál és őrület menedzsment

```
1  async manageDeathAndInsane(type: 'death' | 'insane',
2                                target: Player | NPC) {
3      let status!: ActiveStatus;
4      if (type === 'death') {
5          target.character!.stats.main.hp = 0;
6          target.character!.wounds.large += 1;
7          target.character!.activeStatuses = [];
8          status = {
9              type: StatusType.DEAD,
10             duration: 9999,
11         };
12     } else if (type === 'insane') {
13         target.character!.stats.main.sp = 0;
14         status = {
15             type: StatusType.INSANE,
16             duration: 9999,
17         };
18     }
19     if(status) target.character!.activeStatuses.push(status);
20     target.inCombat = false;
21     this.game!.playerOrder = this.game?.playerOrder
22         .filter(po => po.id !== target.id) ?? [];
```

```

23 let combatEnded = true;
24 this.game!.playerOrder.forEach(po => {
25     const gameMember = this.currentEvent?.NPCs
26         .find(p => p.id === po.id) ??
27         this.game?.camp.raid.find(r => r.id === po.id) ??
28         this.game?.players.find(p => p.id === po.id) ?? null;
29     if (gameMember && gameMember.inCombat)
30         combatEnded = false;
31 });
32 if (combatEnded) {
33     this.game?.players.forEach(p => p.inCombat = false);
34 }
35 try {
36     await this.gameService.updateGame(this.gameId, {
37         players: this.game?.players,
38         camp: this.game?.camp,
39         playerOrder: this.game?.playerOrder,
40         adventure: this.game?.adventure,
41         currentAction: this.game?.currentAction,
42         currentPlayer: this.game?.currentPlayer,
43     });
44 } catch (err) {
45     console.error('Hiba a halál/megőrlés kezelése után a
46                                     mentéskor: ', err);
47 }
48 }

```

AM. kódrészlet. manageDeathAndInsane() függvény.

AN. applyDamage() – Sebzés számítás

```

1 async applyDamage(damage: number, target: Player | NPC) {
2     let character = target.character;
3     if (!character) return;
4     const defValue = character.equipment.armour.defValue;
5     character.stats.main.hp =
6     Math.max(character.stats.main.hp -
7             (Math.max(damage - defValue, 0)), 0);
8     if (character.stats.main.hp === 0) {
9         await this.rollForWound(character, target);
10    }
11 }

```

AN. kódrészlet. applyDamage() függvény.

AO. rollForWound() – Dobás a seb fajtára

```
1  async rollForWound(character: Character,  
2                                target: Player | NPC) {  
3    this.openSnackBar(`${target.name} HP-ja 0-ra csökkent.  
4                                Dobás a sérülés típusára!`);  
5    await this.openDiceRoller(['d6']);  
6    const woundType = () => {  
7        return this.lastRoll <= 4 ? 's' : 'l' }  
8    if (woundType() === 's' && character?.wounds.small! < 2){  
9        this.openSnackBar(`${target.name} egy kis sérülést  
10                                szenvedett el.`)  
11        character.wounds.small += 1;  
12        character.stats.main.hp = character.stats.main.maxHP;  
13        return;  
14    } else if (woundType() === 'l' ||  
15                character.wounds.small! === 2) {  
16        if (character?.wounds.large! < 2) {  
17            if (character.wounds.small! === 2 &&  
18                woundType() === 's') character.wounds.small = 0;  
19            this.openSnackBar(`${target.name} egy nagy sérülést  
20                                szenvedett el.`);  
21            const wound: ActiveStatus = {  
22                type: StatusType.LOST_LIMB,  
23                duration: 9999,  
24            };  
25            character?.activeStatuses.push(wound);  
26            character.wounds.large += 1;  
27            character.stats.main.hp = character.stats.main.maxHP;  
28            const limbsLost = character?.activeStatuses  
29                .filter(s => s.type === StatusType.LOST_LIMB).length ?? 0;  
30            if (limbsLost === 1) {  
31                target.actionsLeft = {  
32                    primary: target.actionsLeft.primary, secondary: false };  
33            }  
34            return;  
35        } else {  
36            this.openSnackBar(`${target.name} túl sok sérülést  
37                                szenvedett el, ezért meghalt.`);  
38            await this.manageDeathAndInsane('death', target);  
39            return;  
40        }  
41    }  
42 }
```

AO. kódrészlet. rollForWound() függvény.

AP. dodgeAttack() – Kikerülés

```
1 async dodgeAttack(damage: number, target: Player | NPC):  
2                               Promise<boolean> {  
3   let character = target.character;  
4   if (!character) return false;  
5   character.stats.main.sp =  
6       Math.max(character.stats.main.sp - 1, 0);  
7   const dmgToDefender = Math.max(damage -  
8       character.equipment. armour.defValue!, 0);  
9   await this.rollCheck(character.stats.physical.dex)  
10  if (this.lastRoll >= 12) {  
11    this.openSnackBar('Sikeres kitérés!');  
12    return true;  
13  }  
14  this.openSnackBar(`Sikertelen kitérés! ${target.name}  
15    ${dmgToDefender > 0 ? `sebzést szenved el: -  
16    ${dmgToDefender} HP` : 'nem szenved el sebzést.'}`);  
17  await this.applyDamage(damage, target);  
18  return false;  
19 }
```

AP. kódrészlet. dodgeAttack() függvény.

AQ. attackBack() – Visszatámadás

```
1 async attackBack(target: Player | NPC, attacker: Player |  
2                               NPC, damage: number): Promise<number> {  
3   if (!this.selectedWeapon) {  
4     localStorage.setItem('rollCheck',this.lastRoll.toString())  
5     throw new Error('Nincs kiválasztva fegyver a  
6       visszatámadáshoz. Válassz fegyvert és próbáld újra.');7   }  
8   target!.character!.stats.main.sp =  
9       Math.max(target!.character!.stats.main.sp - 1, 0);  
10  this.openSnackBar(`Visszatámadás. ${target?.name} sebzést  
11    szenved el: -${Math.max(damage - target?.character?  
12      .equipment. armour.defValue!, 0)} HP`);  
13  await this.applyDamage(damage, target);  
14  this.selectedTarget = attacker;  
15  await this.rollForAttack(this.selectedWeapon.diceCount,  
16      this.selectedWeapon.damage);  
17  const dmgToAttacker = Math.max(this.lastRoll -  
18      attacker.character!.equipment. armour.defValue!, 0);  
19  this.openSnackBar(`Visszatámadás. ${attacker?.name}  
20    ${dmgToAttacker > 0 ? `sebzést szenved el: -  
21    ${dmgToAttacker} HP` : 'nem szenved el sebzést.'}`);
```

```

22     await this.applyDamage(dmgToAttacker, attacker);
23     return dmgToAttacker;
24 }

```

AQ. kódrészlet. attackBack() függvény.

AR. parryAttack() – Hárítás

```

1  async parryAttack(target: Player | NPC,
2  attacker: Player | NPC, damage: number):
3  Promise<{ success: boolean, dmg: number } | null> {
4      if (!this.selectedWeapon) {
5          localStorage.setItem('rollCheck',
6                               this.lastRoll.toString())
7          throw new Error('Nincs kiválasztva fegyver a
8                          visszatámadáshoz. Válassz fegyvert és próbáld újra.');
```

```

41     return { success: false, dmg: newDMG };
42   }
43   return null;
44 }

```

AR. kódrészlet. parryAttack() függvény.

AS. manageActions() – Akció menedzsment

```

1  manageActions(isPrimary: boolean, type: ActionType |
2      MandatoryCampActions | StandardCampActions) {
3      const itemName = this.selectedItem?.name || 'Ismeretlen
4          tárgy';
5      if (this.player) {
6          if (      !this.player.lastAction.performer.id &&
7              !this.player.lastAction.performer.name) {
8              this.player.lastAction.performer = {
9                  id: this.player.id,
10                 name: this.player.name,
11             };
12         }
13         switch (type) {
14             case ActionType.USEITEM:
15                 if (this.game?.camp.isCamping && 'campActionPoints'
16                     in this.player && !this.isRaid()) {
17                     this.player.campActionPoints -= 1;
18                 } else if (isPrimary) {
19                     this.player.lastAction.primary = {
20                         type: type,
21                         item: itemName,
22                     };
23                 } else {
24                     this.player.lastAction.secondary = {
25                         type: type,
26                         item: itemName,
27                     };
28                 }
29                 break;
30             case ActionType.ATTACK:
31                 if (isPrimary) {
32                     this.player.lastAction.primary = {
33                         type: type,
34                         target: this.selectedTarget?.id,
35                         value: this.lastRoll,
36                         reaction: { reacted: false },
37                     };
38                 } else {
39                     this.player.lastAction.secondary = {

```

```

40         type: type,
41         target: this.selectedTarget?.id,
42         value: this.lastRoll,
43         reaction: { reacted: false },
44     };
45 }
46 break;
47 case ActionType.LOOT:
48 case ActionType.TRADE:
49     if (isPrimary) {
50         this.player.lastAction.primary = {
51             type: type,
52             target: this.selectedTarget?.id,
53         };
54     } else {
55         this.player.lastAction.secondary = {
56             type: type,
57             target: this.selectedTarget?.id,
58         };
59     }
60     break;
61 case ActionType.CAMP:
62     if (isPrimary) {
63         this.player.lastAction.primary = {
64             type: type,
65         };
66     } else {
67         this.player.lastAction.secondary = {
68             type: type,
69         };
70     }
71     break;
72 }
73 if ((!this.game?.camp.isCamping || this.isRaid()) &&
74     isPrimary) {
75     this.player.actionsLeft.primary = false;
76 } else if ((!this.game?.camp.isCamping ||
77     this.isRaid()) && !isPrimary) {
78     this.player.actionsLeft.secondary = false;
79 }
80 this.game!.currentAction = this.player?.lastAction;
81 }
82 }

```

AS. kódrészlet. manageActions() függvény.

AT. `cleanUpUserDataOnDelete()` – Akció menedzsment

```
1 export const cleanUpUserDataOnDelete =
2 functions.auth.user().onDelete(async (user) => {
3   const uid = user.uid;
4   try {
5     const batch = db.batch();
6     const userDocRef = db.collection("Users").doc(uid);
7     const userDoc = await userDocRef.get();
8     if (userDoc.exists) {
9       const username = userDoc.data()?.username;
10      if (username) {
11        batch.delete(db.collection("Usernames")
12                      .doc(username));
13      }
14    }
15    batch.delete(userDocRef);
16    const charSnap = await db.collection("Characters")
17      .where("userId", "==", uid).get();
18    charSnap.forEach(doc => batch.delete(doc.ref));
19    const advSnap = await db.collection("Adventures")
20      .where("userId", "==", uid).get();
21    advSnap.forEach(doc => batch.delete(doc.ref));
22    const gameSnap = await db.collection("Games")
23      .where("ownerId", "==", uid).get();
24    gameSnap.forEach(doc => batch.delete(doc.ref));
25    await batch.commit();
26    return null;
27  } catch (error) {
28    console.error("Hiba történt a háttérbeli
29                  takarítás során:", error);
30    return null;
31  }
32 }
33 });
```

AT. kódrészlet. `cleanUpUserDataOnDelete()` függvény.